

ARES 프로젝트 코드 분석 종합 보고서

Autonomous Rover Exploration System — 소스 코드 구조 및 기술 분석

(주)코리아사이언스 / 동명대학교 게임공학과

June 21, 2026

Abstract

본 보고서는 ARES(Autonomous Rover Exploration System) 교육용 로봇 플랫폼의 소스 코드 저장소 (github.com/dknife/ARES2)를 대상으로 한 종합 기술 분석 문서이다. ARES는 초등학생을 대상으로 한 블록 코딩 교육 플랫폼으로, (1) Google Blockly 기반 웹 애플리케이션, (2) 라즈베리파이 피코 MicroPython 펌웨어, (3) 보조 AI 모듈의 세 가지 런타임 도메인으로 구성된다. 본 문서는 하드웨어 구성과 핀 배치, 펌웨어 명령 처리 구조, 웹 서비스 계층, Web Bluetooth 통신, 블록 코딩 구현 방식, WebGL 기반 3D 시뮬레이션, 코드 간 연계 구조를 코드 수준에서 분석하고, 네이티브 앱 패키징 방안과 향후 개선·확장 계획을 제시한다.

Contents

1	프로젝트 개요	2
1.1	프로젝트 정의와 목표	2
1.2	세 가지 런타임 도메인	2
1.3	데이터 흐름 개요	3
1.4	저장소 최상위 구성	3
1.5	보고서 구성	4
2	구성요소 — 하드웨어 및 소프트웨어 요소	4
2.1	시스템 계층 구조	4
2.2	하드웨어 요소	5
2.3	소프트웨어 요소	6
2.3.1	웹 클라이언트 (Web/)	6
2.3.2	피코 펌웨어 (Pico/)	6
2.3.3	외부 라이브러리 및 자산	7
2.4	모듈 활성화 패턴	7
3	사용 하드웨어 — 지원 하드웨어와 연결 핀 번호	8
3.1	GPIO 핀 할당 (pins.py)	8
3.2	지원 하드웨어 사양	9
3.2.1	구동계	10
3.2.2	센서	10

3.2.3	출력 · 표시 · 음향	10
3.3	무선 통신 모듈	10
3.4	핀 · 모듈의 런타임 재구성	11
4	보드 펌웨어 (Pico 기반) — 코드 분석	11
4.1	부팅 및 메인 루프 (main.py)	11
4.2	명령 처리기 (process_data.py)	12
4.2.1	디스패치 구조	12
4.2.2	시스템 · 연결 명령	13
4.2.3	구동 명령	13
4.2.4	출력 · 센서 명령	14
4.3	하드웨어 추상화 (hardware.py)	14
4.4	설정 관리 (system_config.py)	15
4.5	펌웨어 설계상의 제약과 패턴	15
5	서비스 계층 — 웹 애플리케이션 구조	16
5.1	진입점 (HTML)	16
5.2	ES 모듈 의존 구조	16
5.3	상태 · DOM · 상수 관리	17
5.3.1	state.js — 전역 상태	17
5.3.2	elements.js — DOM 참조	17
5.3.3	constants.js — 상수	18
5.4	애플리케이션 컨트롤러 (main.js)	18
5.5	대시보드 (dashboard.html)	18
5.6	어린이 대상 UI 설계	19
6	블루투스 지원 — Web API와 파일 기반 웹앱 제작 시 요구사항	19
6.1	연결 흐름	19
6.2	체크 송신	20
6.3	수신 및 응답 조립	21
6.4	Web Bluetooth API 요구사항	21
6.5	파일 기반(file://) 웹앱 제작 시 요구사항	21
7	코드 블록 구현 방식 — 웹앱 제작 시 요구사항	22
7.1	블록 정의 (blocklyconfig.js)	22
7.2	블록 카테고리(툴박스)	22
7.3	블록 → 명령 변환 (commandexecutor.js)	23
7.3.1	명령 생성: generateCommand(block)	23
7.3.2	값 블록 평가: evaluateValueBlock(block)	24
7.3.3	블록 처리와 제어흐름: processBlock(block)	24
7.3.4	명령 디스패치와 Fire-and-forget	24
7.4	한꺼번에 실행: batch_block	25
7.5	웹앱 제작 시 요구사항	25

8 WebGL 기반 디지털 플랫폼 기술 — 서비스 구조 및 제작 요구사항	25
8.1 three.js 번들링 전략	25
8.2 랜딩 페이지 3D (index.html)	26
8.2.1 file://에서의 모델 로딩	26
8.3 미션 시뮬레이터 (simulation.js)	26
8.4 오프라인 단독 실행본 빌드	27
8.5 웹앱 기반 제작 시 요구사항	28
9 코드 별 연계 구조	29
9.1 종단 간 명령 처리 시퀀스	29
9.2 모듈 간 결합 지점	30
9.3 명령 프로토콜: 두 코드베이스의 계약	30
9.4 상태 공유와 데이터 결합 (웹 내부)	31
9.5 설정의 이중 소유와 정합성	31
9.6 시뮬레이션과 실행의 분기	31
10 네이티브 앱 (macOS, iOS, Android, Desktop) 제작 방식	32
10.1 핵심 제약: 플랫폼별 BLE 지원 차이	32
10.2 데스크톱 (macOS / Windows / Linux)	33
10.2.1 방안 A — Electron (권장)	33
10.2.2 방안 B — Tauri (경량 대안)	33
10.3 Android	33
10.4 iOS	34
10.5 플랫폼별 권장 조합 요약	34
10.6 단계적 도입 로드맵	34
11 주요 기술 요소 및 개선 계획	35
11.1 핵심 기술 요소 요약	35
11.2 개선 계획	35
11.2.1 1. 통신 계층 추상화	35
11.2.2 2. 명령 프로토콜의 단일 출처화	35
11.2.3 3. 펌웨어 비차단화	36
11.2.4 4. 설정 정합성 관리	36
11.2.5 5. 안전성 · 견고성	36
11.2.6 6. AI 보조 기능 통합	36
11.2.7 7. 접근성 · 다국어	36
11.2.8 8. 테스트 · 빌드 자동화	37
12 기술적 파급효과와 향후 추가 개발 계획	37
12.1 기술적 파급효과	37
12.1.1 교육적 파급효과	37
12.1.2 기술적 파급효과	37
12.2 향후 추가 개발 계획	38
12.2.1 단기 (기존 자산 강화)	38
12.2.2 중기 (플랫폼 확장)	38

12.2.3 장기 (플랫폼 고도화)	38
12.3 개발 로드맵 개관	38
12.4 결론	39
13 코드 분석과 공동 개발 가이드	39
13.1 코드 분석 진입점과 권장 읽기 순서	39
13.2 개발 환경 구축	39
13.2.1 웹 UI	39
13.2.2 피코 펌웨어	40
13.2.3 AI 모듈	40
13.3 변경 영향 지도 — 작업별 수정 파일	40
13.4 디버깅 · 로깅 · 진단	40
13.5 오프라인 빌드 · 배포와 기기 간 동기화	41
13.6 공동 개발 분석 체크리스트	41
14 연구 주제 도출과 논문 방향	42
14.1 학술적 가치의 세 축	42
14.2 연구 요소 1 — 시뮬레이터-실기 동형 설계의 교육 효과	42
14.3 연구 요소 2 — 오프라인 우선 웹-임베디드 교구 아키텍처	43
14.4 연구 요소 3 — 브라우저-임베디드(Web Bluetooth) 통신 특성	43
14.5 연구 요소 4 — 제약 환경 모듈식 펌웨어와 결함 내성	43
14.6 연구 요소 5 — 오프라인 · 경량 자연어 → 블록 변환	44
14.7 인접 연구 주제(HCI)	44
14.8 논문화 로드맵	44
14.9 통합 논문 전략	44
14.10 연구 타당성과 윤리	45
A1 Pico 소스 코드별 API Reference	46
A1.1 main.py	46
A1.2 process_data.py	46
A1.3 hardware.py	47
A1.4 system_config.py	48
A1.5 pins.py	48
A1.6 wheel.py	48
A1.7 dcmotor.py	49
A1.8 buzzer.py	49
A1.9 leds.py	50
A1.10 gun.py	50
A1.11 ultrasonic.py	50
A1.12 magsensor.py	50
A1.13 ssd1306.py	51
A1.14 icon.py	51

A2 Web 소스 코드별 API Reference	52
A2.1 constants.js	52
A2.2 state.js	52
A2.3 elements.js	52
A2.4 logger.js	52
A2.5 romanize.js	53
A2.6 blocklyconfig.js	53
A2.7 bluetooth.js	53
A2.8 commandexecutor.js	54
A2.9 ai_helper.js	54
A2.10ai_helper_grammar.js	54
A2.11 simulation.js	55
A2.12main.js	55
A2.13mobile-preview.js	55
 A3 AI 소스 코드별 API Reference	 55
A3.1 ai.py	55

1 프로젝트 개요

1.1 프로젝트 정의와 목표

ARES(Autonomous Rover Exploration System)는 초등학생을 주 대상으로 하는 교육용 로봇 코딩 플랫폼이다. 학습자는 웹 브라우저에서 Google Blockly¹ 기반의 시각적 블록을 조립하여 프로그램을 작성하고, 이를 Bluetooth로 라즈베리파이 피코(Raspberry Pi Pico) 기반 로버 “알비(Albi)”에 전송하여 실제 하드웨어를 제어한다. “화성 탐사 로버”라는 서사를 통해 순차 실행, 반복, 조건, 변수 등 프로그래밍의 기초 개념을 단계적으로 학습하도록 설계되었다.

프로젝트 핵심 특징

- **대상:** 초등학교 저·고학년 (블록 코딩 입문 학습자)
- **언어:** UI 텍스트, 코드 주석, 문서 모두 한국어
- **제어 대상:** 라즈베리파이 피코(RP2040) + BLE UART 모듈(HM-10/BT05)
- **배포 방식:** 웹(HTTPS) 호스팅 및 오프라인 file:// 단독 실행 모두 지원

1.2 세 가지 런타임 도메인

ARES는 역할이 명확히 분리된 세 개의 실행 영역으로 구성된다.

1. **웹 UI (Web/)** — 정적 HTML/JS로 작성된 브라우저 애플리케이션. Google Blockly로 블록 코딩 편집기를 제공하고, Web Bluetooth² API로 로버와 통신한다. ES 모듈³(import/export) 구조이며 진입점은 main.html(블록 편집기), dashboard.html(시스템 모니터링), index.html(3D 랜딩 페이지)이다.
2. **피코 펌웨어 (Pico/)** — 라즈베리파이 피코에서 동작하는 MicroPython⁴ 코드. main.py가 UART⁵를 통해 BLE⁶ 명령을 수신하면 CommandProcessor(process_data.py)가 이를 해석하여 하드웨어 모듈로 분배한다. 모든 하드웨어는 싱글턴⁷ robot (hardware.py)을 통해 접근한다.
3. **AI 모듈 (AI/, Web/ai_helper*.js)** — 자연어를 블록으로 변환하는 보조 기능. 로컬 LLM⁸ 기반 PoC⁹(AI/ai.py)와, 브라우저에서 동작하는 문법 기반 자연어→블록 파서 (Web/ai_helper.js, ai_helper_grammar.js)로 구성된다.

¹**Blockly:** Google이 개발한 오픈소스 시각적 프로그래밍 라이브러리. 퍼즐 형태의 블록을 끼워 맞춰 프로그램을 구성하며, 조립된 블록은 실행 가능한 코드나 명령으로 변환된다.

²**Web Bluetooth:** 웹 브라우저에서 자바스크립트로 BLE 기기와 직접 연결·통신할 수 있게 하는 표준 웹 API. 보안상 HTTPS 또는 localhost 등 보안 컨텍스트에서만 동작한다.

³**ES 모듈(ECMAScript Module):** 자바스크립트의 표준 모듈 시스템. import/export로 코드를 파일 단위로 분리하고 서로 가져다 쓸 수 있게 하여, 큰 프로그램을 책임별로 나눈다.

⁴**MicroPython:** 마이크로컨트롤러처럼 메모리·성능이 제약된 환경에서 동작하도록 만든 경량 파이썬 구현. 표준 파이썬 문법의 부분집합을 지원한다.

⁵**UART(Universal Asynchronous Receiver/Transmitter):** 두 장치가 별도의 클럭 신호 없이 약속된 속도(보율, baud)로 한 바이트씩 직렬로 주고받는 비동기 시리얼 통신 방식.

⁶**BLE(Bluetooth Low Energy):** 저전력 블루투스. 적은 전력으로 소량의 데이터를 주고받도록 설계된 근거리 무선 통신 규격.

⁷**싱글턴(Singleton):** 클래스의 인스턴스를 프로그램 전체에서 단 하나만 생성해 공유하도록 강제하는 설계 패턴. 하드웨어처럼 실체가 하나뿐인 자원을 다룰 때 쓴다.

⁸**LLM(Large Language Model, 대규모 언어 모델):** 방대한 텍스트로 학습되어 자연어 질문에 답하거나 문장을 생성하는 인공지능 모델.

⁹**PoC(Proof of Concept, 개념 증명):** 아이디어의 실현 가능성을 보이기 위한 시험적·초기 구현.

1.3 데이터 흐름 개요

전체 시스템의 명령 흐름은 다음과 같다. 웹 UI에서 생성된 텍스트 명령은 Web Bluetooth를 거쳐 피코의 UART로 전달되고, 펌웨어가 이를 해석하여 하드웨어를 구동한 뒤 응답을 역방향으로 반환한다.

Listing 1. 명령 데이터 흐름

```

웹
[ UI: Blockly 블록]
  | commandexecutor.js -> 명령 문자열 생성 예(: "SERVO_tFORWARD,2")
  v
[bluetooth.js: Web Bluetooth]
  | UART 서비스(0xFFE0) / 바이트20 체크 / 개행 중단
  v
[HM-10 / BT05 BLE 모듈] --(UART 9600 baud)--> 피코[]
  v
[main.py: UART 수신 루프] -> 버퍼링개행까지() -> CommandProcessor
  v
[process_data.py] -> 하드웨어 모듈(wheel/leds/buzzer/sensor/oled ...)
  v응답예
(: "DIST:23.5") -> BLE 바이트20 체크 -> 웹 UI 로그변수/ 갱신
  
```

이 흐름을 도식화하면 그림 1와 같다. 세 도메인은 서로 다른 언어(JavaScript, MicroPython)와 실행 환경에서 동작하지만, 개행으로 구분되는 텍스트 명령 프로토콜이라는 단일 계약으로 느슨하게 결합된다. 이러한 “공유 코드 없는 텍스트 계약” 구조는 양측을 독립적으로 개발·교체할 수 있게 하는 동시에, 한쪽의 프로토콜 변경이 반드시 다른 쪽에 반영되어야 하는 의존을 만든다(상세는 9장 참조).

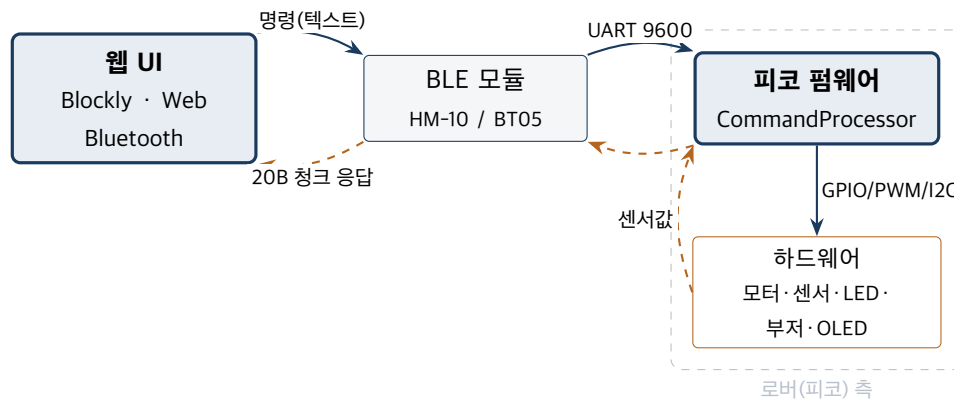


Figure 1. ARES 시스템 개요 및 데이터 흐름 연계도

1.4 저장소 최상위 구성

경로	역할
Web/	Blockly 블록 편집기, Web Bluetooth 통신, WebGL 시뮬레이터, 정적 자산
Pico/	MicroPython 펌웨어(부팅 · 명령 처리 · 하드웨어 드라이버 · 설정)
AI/	로컬 LLM 기반 코드 도우미 PoC (ai.py)

경로	역할
Document/	API 문서, 블록 가이드, 모듈 관리, 빌드 가이드, 본 분석 보고서
Missions/	12차시 미션 교재 LaTeX 소스 및 PDF
Papers/	연구 논문 · 발표 자료
KSWeb/	배포/아카이브용 웹 산출물
XML Presets/	예제 Blockly 프로그램(XML)
build.ps1 / .sh / .bat	오프라인 단독 실행본 빌드 자동화 스크립트

1.5 보고서 구성

본 보고서는 다음 순서로 전개된다. 2장에서 하드웨어·소프트웨어 구성요소를 조감하고, 3~4장에서 하드웨어 핀 배치와 피코 펌웨어를 코드 수준에서 분석한다. 5~8장은 웹 서비스 계층, Web Bluetooth, 블록 코딩 구현, WebGL 디지털 플랫폼을 다루며, 각 장은 “파일 기반 웹앱 제작 시 요구사항”을 함께 정리한다. 9장은 코드 간 연계 구조를, 10장은 네이티브 앱 패키징 방안을, 11~12장은 개선 계획과 기술적 파급효과 및 향후 개발 계획을 제시한다.

주요 분석 요소 — 프로젝트 전체 그림 읽기

설계 의도

- ARES는 웹·펌웨어·AI 세 영역을 텍스트 명령이라는 단순한 약속만으로 연결한다. 서로의 내부 구현을 몰라도 되게 하여 독립적 개발·교체가 쉽다.
- 인터넷·설치가 어려운 교실을 전제로 오프라인 우선으로 설계했다.

분석할 때 핵심 질문

- 블록 하나를 누르면 명령은 어디서 만들어져 어떤 경로로 로버에 도달하는가?
- 웹과 펌웨어가 코드를 공유하지 않는데 어떻게 서로 통하는가?

점검 요소

- 좋아하는 블록 1개를 골라 “블록 → 명령 문자열 → BLE → 펌웨어 핸들러 → 하드웨어”의 전 경로를 직접 그려 보고, 2장 이후 내용과 대조하라.

2 구성요소 — 하드웨어 및 소프트웨어 요소

본 장은 ARES 시스템을 이루는 하드웨어 요소와 소프트웨어 요소를 조감한다. 세부 분석은 3장(하드웨어 핀)과 4장(펌웨어), 5~8장(웹 계층)에서 이어진다.

2.1 시스템 계층 구조

ARES는 “웹 클라이언트 — 무선 링크 — 임베디드 펌웨어 — 물리 하드웨어”의 4계층으로 볼 수 있다.

계층	구성요소	핵심 기술
프레젠테이션	블록 편집기, 대시보드, 3D 랜딩/시뮬레이터	HTML5, CSS, Google Blockly, three.js
애플리케이션	명령 생성기, 상태 관리, 로거, AI 도우미	ES 모듈 JavaScript
통신	Web Bluetooth ↔ BLE UART	GATT UART 서비스(0xFFE0/0xFFE1), 20바이트 청크
펌웨어	명령 처리기, 하드웨어 추상화, 설정	MicroPython, 싱글턴 패턴
물리	모터/센서/표시/음향/발사	RP2040 GPIO, PWM, I2C, ADC

각 계층은 바로 아래 계층에만 의존하며, 인접하지 않은 계층과 직접 결합하지 않는다. 예컨대 프레젠테이션 계층의 블록은 통신 계층의 BLE 세부를 알지 못하고, 통신 계층은 펌웨어 핸들러의 구현을 알지 못한다. 이 단방향 의존(그림 2)은 계층별 교체·시험을 쉽게 한다. 특히 통신 계층은 “전송 가능한 텍스트 명령”만 다루므로, 실기(BLE) 대신 시뮬레이터(3D)로 대상을 바꿔도 상위 계층이 동일하게 동작한다(9장 시뮬레이션 분기 참조).

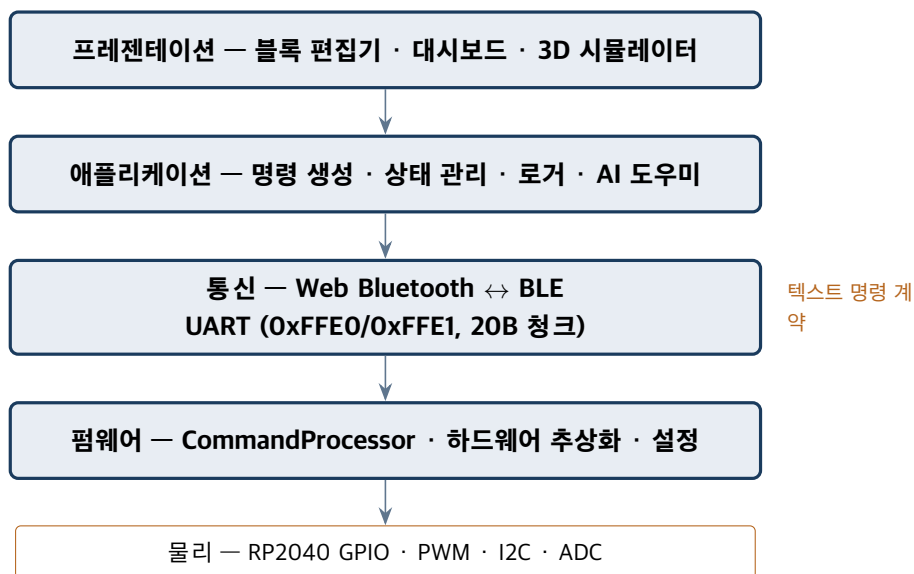


Figure 2. 4계층 단방향 의존 구조

2.2 하드웨어 요소

로버에 탑재되는 하드웨어 모듈은 모두 선택적이며, 펌웨어 설정 파일 `system.json`을 통해 개별적으로 활성화·비활성화할 수 있다.

모듈	드라이버 클래스	역할
서보 휠	KSWheel	좌·우 연속회전 서보로 전진/후진/회전 (차동 구동)
DC 모터	KSDCMotor	메인 구동 모터, PWM H-브리지 속도 제어
부저	KSBuzzer	PWM 톤 생성, 비차단(non-blocking) 재생, 멜로디 지원

모듈	드라이버 클래스	역할
LED 배열	KSLEDs	6개 LED, PWM 밝기 제어(0.0~1.0), 스와이프 효과
거리 센서	KSDistance	HC-SR04 초음파, cm 단위 거리 측정
자기 센서	KSMagSensor	자기장 검출(디지털, 폴업)
발사기	KSGun	BB탄 발사 모터, 소프트 스타트
OLED	SSD1306_I2C	128×64 I2C 디스플레이, 텍스트/아이콘 출력

2.3 소프트웨어 요소

2.3.1 웹 클라이언트 (Web/)

ES 모듈로 분리된 JavaScript 파일들이 명확한 책임 경계를 가진다.

파일	역할
main.js	앱 초기화, 라우팅, Blockly 워크스페이스 구성, 차시/미션 로딩
blocklyconfig.js	커스텀 블록 정의 및 카테고리(툴박스)
commandexecutor.js	블록 → 명령 문자열 변환 및 전송, 제어흐름 처리
bluetooth.js	BluetoothManager: BLE 연결, 청크 송수신, 응답 처리
constants.js	UUID, 타이밍 상수, 기본 시스템 설정
state.js	전역 상태(연결, 변수, 실행 플래그)
elements.js	DOM 요소 참조 캐시
logger.js	통신 로그 UI
simulation.js	three.js 기반 3D 시뮬레이터
ai_helper.js, ai_helper_grammar.js	자연어 → 블록 변환(문법 기반)
romanize.js	한글 → 로마자 변환(OLED 텍스트용)
mobile-preview.js	모바일 프레임 미리보기

2.3.2 피코 펌웨어 (Pico/)

파일	역할
main.py	부팅 시퀀스, UART 수신 메인 루프, 응답 전송
process_data.py	CommandProcessor: 명령 파싱 및 하드웨어 분배
hardware.py	RobotHardware 싱글톤: 모듈 초기화/수명 관리
system_config.py	SystemConfig: system.json 입출력, 설정 관리

파일	역할
pins.py	GPIO 핀 중앙 정의
wheel.py, dcmotor.py, buzzer.py, leds.py, gun.py	구동/출력 드라이버
ultrasonic.py, magsensor.py	센서 드라이버
ssd1306.py, icon.py	OLED 드라이버 및 아이콘 비트맵
system.json	모듈 활성화 플래그, 핀 할당, 보정값 저장

2.3.3 외부 라이브러리 및 자산

- **Google Blockly** (v11) — 시각적 블록 코딩 엔진. 오프라인 빌드 시 vendor/에 로컬 사본으로 동봉.
- **three.js**¹⁰ — WebGL¹¹ 3D 렌더링. vendor/three-bundle.min.js로 사전 번들(로컬, CDN 비의존).
- **meshopt decoder**¹² — EXT_meshopt_compression으로 압축된 GLB¹³ 모델 디코딩.
- **GLB 3D 모델** (Web/Mesh/) — 로버, 알비, 신호등, 발사대 등 미션 환경 모델.

2.4 모듈 활성화 패턴

모든 하드웨어 모듈은 동일한 안전 초기화 패턴을 따른다. 이는 일부 부품이 없거나 고장 난 상태에서도 시스템 전체가 부팅되도록 보장한다.

Listing 2. hardware.py의 모듈 초기화 패턴

```
if sys_config.is_module_enabled('wheel'):
    try:
        from wheel import KSWheel
        self.wheel = KSWheel()
    except Exception as e:
        print("Wheel init failed:", e)
        self.wheel = None # 실패 시 None -> 사용 측에서 건너뛸
```

명령 처리 측(process_data.py)은 사용 전 if not robot.wheel: return 0와 같이 모듈 존재 여부를 확인하므로, 비활성 모듈에 대한 명령은 조용히 무시된다. 이 “설정 검사 → 안전 초기화 → 사용 측 널 검사”의 3단 방어(그림 3)가 부분 고장·부분 구성 상황에서도 시스템 부팅과 운용을 보장하는 핵심 패턴이다.

¹⁰**three.js**: 웹 브라우저에서 WebGL을 쉽게 다루도록 해주는 자바스크립트 3D 그래픽 라이브러리.

¹¹**WebGL(Web Graphics Library)**: 웹 브라우저에서 별도 플러그인 없이 GPU 가속으로 3D·2D 그래픽을 렌더링하는 표준 API.

¹²**meshopt**: 3D 메시(점·면) 데이터를 작게 압축하고 빠르게 복원하는 기법·라이브러리. 모델 파일 용량을 줄여 로딩을 앞당긴다.

¹³**GLB**: glTF 3D 모델을 하나의 바이너리 파일로 담은 포맷. 형상·재질·애니메이션을 함께 포함한다.

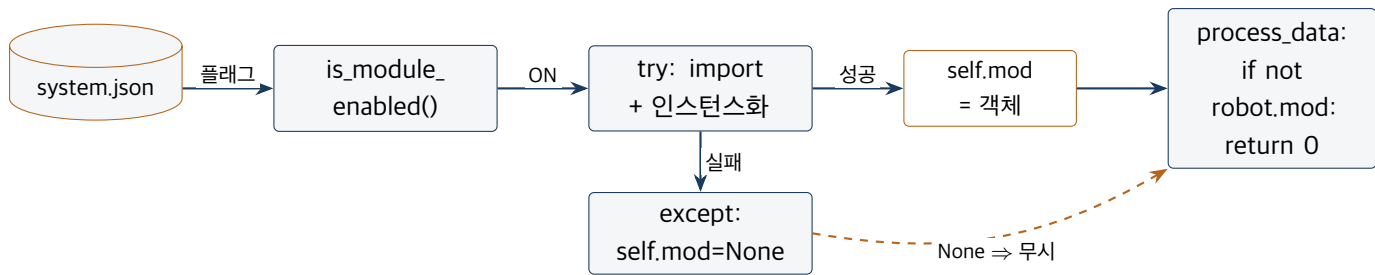


Figure 3. 모듈 활성화 · 안전 초기화 연계 흐름

주요 분석 요소 — 선택적 모듈과 안전 초기화

설계 의도

- 모든 하드웨어를 선택적으로 만들어, 부품 구성이 다른 로버도 같은 펌웨어로 돌린다.
- “설정 검사 → try/except 초기화 → 사용 측 None 검사”의 3단 방어로 일부 부품이 없거나 고장 나도 전체가 부팅된다.

분석할 때 핵심 질문

- 어떤 모듈이 빠졌을 때 시스템은 멈추는가, 무시하는가? 왜 그렇게 했는가?
- 이 3단 방어가 없다면 어떤 문제가 생길까?

점검 요소

- system.json에서 한 모듈을 꺼 보고, 관련 명령이 어떻게 조용히 무시되는지 직렬 로그로 확인하라.

3 사용 하드웨어 — 지원 하드웨어와 연결 핀 번호

본 장은 Pico/pins.py와 Pico/system_config.py에 정의된 실제 GPIO¹⁴ 핀 할당을 정리한다. 모든 핀은 라즈베리파이 피코(RP2040¹⁵)의 GP 번호 기준이다.

3.1 GPIO 핀 할당 (pins.py)

기능	상수명	GPIO
UART 송신(블루투스)	UART_TX_PIN	GP0
UART 수신(블루투스)	UART_RX_PIN	GP1
DC 모터 방향	DCMOTOR_DIR_PIN	GP8
DC 모터 PWM	DCMOTOR_PWM_PIN	GP9
I2C SDA(OLED)	I2C_SDA_PIN	GP4
I2C SCL(OLED)	I2C_SCL_PIN	GP5

¹⁴**GPIO(General-Purpose Input/Output, 범용 입출력)**: 마이크로컨트롤러의 디지털 신호 입출력 핀. 소프트웨어로 켜고 끄거나 (출력) 외부 신호를 읽을(입력) 수 있다.

¹⁵**RP2040**: 라즈베리파이 재단이 설계한 마이크로컨트롤러 칩. 라즈베리파이 피코에 탑재되며 듀얼 코어 ARM Cortex-M0+ 기반이다.

기능	상수명	GPIO
초음파 트리거	ULTRASONIC_TRIG_PIN	GP6
초음파 에코	ULTRASONIC_ECHO_PIN	GP7
서보 휠(우)	SERVO_RIGHT_PIN	GP12
서보 휠(좌)	SERVO_LEFT_PIN	GP13
부저	BUZZER_PIN	GP15
LED 배열(6개)	LED_PINS	GP21, GP20, GP19, GP18, GP17, GP16
BB탄 발사기	GUN_PIN	GP22
자기장 센서	MAGSENSOR_PIN	GP26
예비(미사용)	RESERVED_PINS	GP10, GP11, GP14, GP27, GP28

LED_PINS는 [21, 20, 19, 18, 17, 16] 순서로, LED0(메인)부터 LED5(오른쪽 끝)까지 6개이다. 명령에서 LED 번호는 0~5로 지정한다.

핀과 주변장치의 물리적 연결을 한눈에 보면 그림 4과 같다. RP2040의 GPIO는 인터페이스 유형에 따라 묶이는데, UART(GP0/1)는 BLE 통신, I2C(GP4/5)는 OLED, 나머지는 PWM(서보·DC·LED·부저·발사기)과 디지털 I/O(초음파·자기)로 사용된다. GP10·11·14·27·28은 예비로 남겨 확장 모듈에 대비한다.

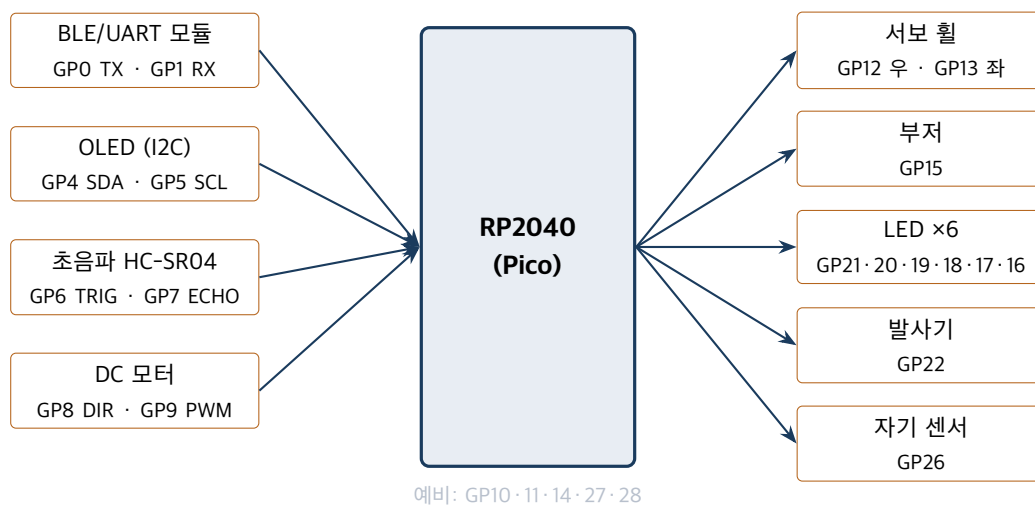


Figure 4. RP2040 핀 · 주변장치 연결 연계도

3.2 지원 하드웨어 사양

각 부품은 RP2040의 GPIO에 연결되어 PWM¹⁶, I2C¹⁷, ADC¹⁸ 등 하드웨어 인터페이스로 제어된다. 모터처럼 회전 방향 전환이 필요한 경우 H-브리지¹⁹ 회로를 사용한다.

¹⁶**PWM(Pulse Width Modulation, 펄스 폭 변조)**: 일정 주기에서 신호가 켜져 있는 시간 비율(듀티)을 조절해 평균 전력을 바꾸는 기법. 모터 속도·LED 밝기 제어에 쓴다.

¹⁷**I2C**: 데이터선(SDA)과 클록선(SCL) 두 가닥으로 여러 장치를 연결하는 직렬 통신 버스. 본 프로젝트에서는 OLED 연결에 사용한다.

¹⁸**ADC(Analog-to-Digital Converter, 아날로그-디지털 변환기)**: 연속적인 아날로그 전압을 디지털 수치로 변환하는 회로.

¹⁹**H-브리지**: 네 개의 스위치로 모터에 흐르는 전류 방향을 바꿔 정·역 회전을 가능하게 하는 회로.

3.2.1 구동계

부품	사양 및 제어 방식
서보 휠	연속회전 서보 2개. PWM 50 Hz. 중립 듀티 약 5000(1.5 ms), 최소 1000~최대 9000 범위. 좌우 보정 계수(left_calibration, right_calibration)로 직진성 교정.
DC 모터	H-브리지 구동(GP8 방향, GP9 PWM). PWM 약 20 kHz, 듀티 0~65535. 브레이크 변조 방식으로 양방향 대칭 토크 확보.

3.2.2 센서

부품	사양 및 측정 방식
초음파 거리	HC-SR04. TRIG(GP6) 펄스 후 ECHO(GP7) 에코 시간 측정, distance_cm = (duration/2) * 0.0343로 환산.
자기 센서	디지털 입력(GP26), 풀업. 자석 근접 시 0(액티브 로우)을 1로 변환하여 검출.
온도	RP2040 내장 온도 센서(ADC4). 상태 보고용.

3.2.3 출력 · 표시 · 음향

부품	사양 및 제어 방식
LED 배열	6개 PWM 출력(약 20 kHz). 밝기 0.0~1.0을 듀티 0~65535로 매핑. 스와이프 애니메이션 지원.
부저	PWM 톤(주파수 가변). 비차단 재생: 명령은 즉시 반환되고 메인 루프의 buzzer.update()가 지속시간 경과 후 자동 정지.
OLED	SSD1306, 128×64 모노크롬, I2C(0) 400 kHz, 주소 0x3c. 텍스트/사각형/아이콘(로버·화성·눈 뜬/감은) 출력.
발사기	BB탄 발사 모터(GP22). PWM 약 1kHz. 소프트 스타트 후 단발 발사 (fire_once()).

3.3 무선 통신 모듈

피코는 UART(GP0/GP1)를 통해 BLE UART 브리지 모듈(HM-10 또는 BT05, CC2541 계열 Bluetooth 4.0)과 연결된다. 통신 파라미터는 다음과 같다.

- UART: UART(0), **9600 baud**, 8N1
- 프레임 종단: 개행 문자(\n)

- BLE GATT²⁰ UART 서비스 UUID²¹: 0000ffe0-0000-1000-8000-00805f9b34fb
- BLE 특성(characteristic) UUID: 0000ffe1-0000-1000-8000-00805f9b34fb
- 장치 이름 최대 길이: **10자** (BT05/HMSOft 제약)

3.4 핀 · 모듈의 런타임 재구성

핀 할당과 모듈 활성 상태는 펌웨어 재기록 없이 변경할 수 있다.

- SET_PIN, 핀이름, 번호 — 핀 번호를 system.json에 저장. 유효 범위 0~28. 다음 부팅 시 적용.
- SET_MODULE, 모듈명, 0|1 — 모듈 활성/비활성 토글.
- GET_MODULES — 전체 모듈 활성 상태 조회.

설계 의도

핀과 모듈을 pins.py와 system.json에 중앙 집중화함으로써, 교실 현장에서 부품 구성이 다른 로버(예: 발사기 미장착)도 동일한 펌웨어로 운용할 수 있다. 운영자는 코드를 고치지 않고 설정만으로 하드웨어 구성을 맞춘다.

주요 분석 요소 — 핀과 인터페이스의 중앙 관리

설계 의도

- 핀 번호를 pins.py 한 곳에 모으고 런타임 재구성(SET_PIN)을 지원해, 코드를 고치지 않고 하드웨어 구성을 바꾼다.
- PWM · I2C · ADC 같은 인터페이스를 부품 성격에 맞게 골라 썼다.

분석할 때 핵심 질문

- 핀 번호가 코드 곳곳에 흩어져 있었다면 어떤 불편이 생길까?
- 모터엔 PWM, OLED엔 I2C, 온도엔 ADC를 쓴 이유는 무엇인가?

점검 요소

- LED 핀 하나를 예비 핀으로 옮기고(설정 변경 후 재부팅) 정상 동작을 확인하라.

4 보드 펌웨어 (Pico 기반) — 코드 분석

본 장은 Pico/ 디렉터리의 MicroPython 펌웨어를 부팅 흐름, 명령 처리, 하드웨어 추상화, 설정 관리의 네 측면에서 분석한다.

4.1 부팅 및 메인 루프 (main.py)

펌웨어 진입점은 AresRover 클래스이다. 생성자에서 UART(0) (9600 baud, GP0/GP1)와 CommandProcessor, 하드웨어 싱글턴 robot을 초기화한다. run()은 부팅 시퀀스 후 메인 루프를 무한 반복한다.

Listing 3. 메인 루프 골격

²⁰GATT(Generic Attribute Profile): BLE에서 데이터를 서비스와 특성(characteristic) 단위로 구조화해 주고받도록 정의한 프로토콜.

²¹UUID(범용 고유 식별자): 중복 가능성이 거의 없는 128비트 식별자. BLE 서비스 · 특성을 구분하는 데 쓴다.

```
def run(self):
    self.boot() # 안전 정지, 부팅음, OLED "ARES READY"
    while self.is_running:
        line = self._read_uart_line() # 개행까지 버퍼링타임아웃( 내)
        if line and not self._is_status_message(line): # '+' 상태 메시지 필터
            resp = self._process_command(line)
            if self._needs_response(line):
                self._send_response(resp) # 바이트20 청크 전송
            self.robot.buzzer.update() # 비차단 부저 자동 정지
            time.sleep_ms(MAIN_LOOP_DELAY_MS) # 약 10ms 주기
```

핵심 동작 특성:

- **다중 청크 명령 조립**: 수신 버퍼(MAX_BUFFER_SIZE, 512바이트)에 바이트를 누적하고 개행 종단을 폴링한다. 여러 BLE 청크에 걸쳐 도착하는 긴 명령(BATCH, LED 패턴, SYS_SET)을 위해 라인 완성까지 타임아웃(RECEIVE_TIMEOUT_MS, 약 500 ms)을 둔다.
- **상태 메시지 필터**: +로 시작하는 BLE 모듈 상태 메시지를 무시한다.
- **Fire-and-forget**: 응답을 보내지 않는 명령 집합(NO_RESPONSE_CMDS)을 두어, 웹 프론트엔드와의 명령-응답 오정렬을 방지한다.
- **비차단 부저**: 음향 명령은 즉시 반환되고, 루프마다 호출되는 buzzer.update()가 경과 시간을 검사해 자동 정지시킨다.

한 번의 루프 반복에서 명령이 거치는 수명주기는 그림 5과 같다. 수신·필터·처리·응답·부저 갱신의 5 단계가 약 10 ms 주기로 반복되며, 응답 여부 판단(_needs_response)이 웹과의 명령-응답 정합성을 지키는 분기점이다.

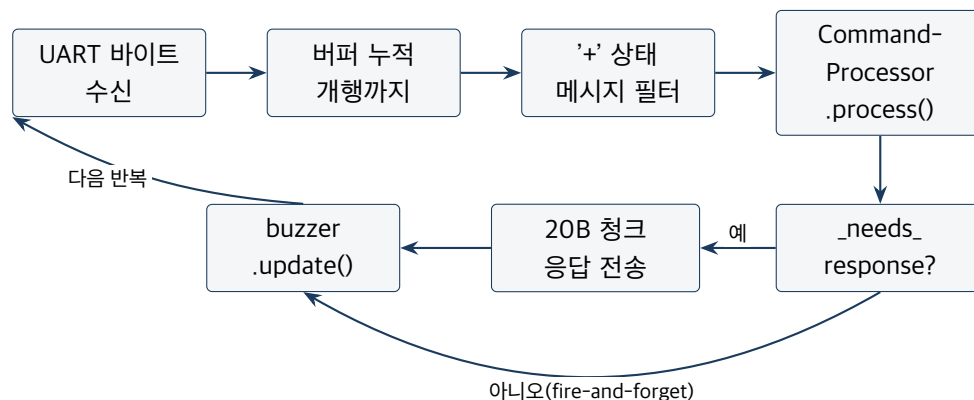


Figure 5. 펌웨어 메인 루프 명령 수명주기

4.2 명령 처리기 (process_data.py)

CommandProcessor.process(data)는 수신 문자열을 파싱하여 적절한 핸들러로 분배한다. 명령은 심표로 인자를 구분하는 텍스트 프로토콜이며, 성공 시 1, 미인식·오류 시 0 또는 ERROR를 반환한다.

4.2.1 디스패치 구조

process()는 긴 if/elif 사슬로, 두 가지 매칭 방식을 혼용한다. 인자 없는 명령은 완전 일치(data == "PING")로, 인자 있는 명령은 접두 일치(data.startswith("SYS_SET"))로 판별한다.

Listing 4. process() 디스패치 사슬(발췌)

```

if data == "PING":          return self._handle_ping()
elif data == "STOP_ALL":    return self._handle_stop_all()
elif data.startswith("SYS_SET"): return self._handle_sys_set(data)
elif data.startswith("BATCH;"): return self._handle_batch(data)
elif data.startswith("SERVO_tFORWARD"): return self._handle_timed_wheel(data, "forward")
elif data.startswith("tFORWARD"): return self._handle_timed_wheel(data, "forward")
...
else: return 0 # 미인식 명령

```

접두 일치의 순서 의존성

접두 일치는 **선언 순서가 중요하다**. 더 구체적인 접두(SERVO_tFORWARD)를 더 일반적인 접두(tFORWARD)보다 먼저 검사해야 한다. 또한 startswith("PING")처럼 짧은 접두는 다른 명령과의 우연한 일치를 피하도록 배치된다. 새 명령을 추가할 때는 이 순서 규칙을 지켜야 한다.

4.2.2 시스템 · 연결 명령

명령	동작
PING	연결 확인. OLED에 "CONNECTED!"와 장치명 표시, 1 반환
STOP_ALL	비상 정지. 모든 모터/출력 정지, OLED "EMERGENCY STOP"
GET_STATUS	STATUS, 거리, 자기, 온도, 여유메모리, 업타임 반환
GET_SYS	SYS_VALUES, 속도, 충돌거리, 자동정지, 이름, 좌보정, 우보정 반환
GET_MODULES	MODULES, 모듈:ON OFF, ... 형식으로 전체 모듈 상태 반환
SYS_SET, 속도, 거리, 정지, 이름	시스템 설정 갱신 후 저장
SET_PIN, 이름, 번호	핀 할당 변경
SET_MODULE, 이름, 0 1	모듈 활성화/비활성
BATCH;cmd1 cmd2 ...	여러 명령 순차 실행(단일 응답)
CALIB_START / CALIB_SET, 좌, 우	휠 직진 보정 시작/적용
SLEEP, 초	지정 시간 대기(차단)

4.2.3 구동 명령

명령	동작
SERVO_FORWARD / FORWARD 등	서보 휠 연속 전진/후진/좌/우회전
SERVO_STOP / STOP	서보 휠 정지
tFORWARD, 초 등	서보 휠 시간 지정 이동(차단), 완료 후 응답
DC_FORWARD / MAIN_FORWARD 등	DC 모터 연속 구동
DC_STOP	DC 모터 정지
DC_tFORWARD, 초 등	DC 모터 시간 지정 구동(차단)

서보 휠은 KSWheel이 좌우 서보 각도를 제어하여 전진/후진/회전을 구현하며, 보정 계수를 곱해 직진성을 교정한다. DC 모터는 max_speed 설정값을 기준으로 PWM 듀티를 산출한다.

4.2.4 출력 · 센서 명령

명령	동작
LED_ON, 번호, 밝기	단일 LED 켜기(밝기 0.0~1.0)
LED_OFF, 번호 ALL	단일/전체 LED 끄기
[v0 v1 v2 v3 v4 v5]	6개 LED 밝기 일괄 설정(공백 구분)
LED_PATTERN	스와이프 애니메이션
BUZZER_ON, 주파수, 지속	비차단 톤 시작
SING	내장 멜로디 재생
MSG, 텍스트 / MSG_XY, x, y, 텍스트	OLED 텍스트 출력
ICON, 이름, x, y	아이콘 표시(rover/mars/open_eye/closed_eye)
CLEAR_DISPLAY / CLEAR_RECT, x, y, w, h	화면/영역 지우기
GUN_FIRE	BB탄 단발 발사
DISTANCE	DIST: 값 반환(cm)
MAGNET	MAG: 0 1 반환

4.3 하드웨어 추상화 (hardware.py)

RobotHardware는 싱글턴 패턴(_instance)으로 구현되어 전역 robot 인스턴스로 접근된다. 각 모듈 속성은 초기값 None이며, system_config.is_module_enabled() 검사 후 try-except로 초기화한다. DC 모터는 안전을 위해 가장 먼저 초기화하고 즉시 정지시킨다.

주요 메서드:

- get_temperature() — ADC4 기반 섭씨 온도 환산

- `get_status_dict()` — 거리·자기·온도·여유메모리(`gc.mem_free()`)·업타임 디셔너리
- `stop_all()` — 모듈별 예외 처리를 동반한 전체 안전 정지

4.4 설정 관리 (`system_config.py`)

`SystemConfig`(싱글턴)는 `DEFAULT_CONFIG`를 기준으로 두고, `system.json`이 존재하면 병합하여 덮어쓴다. 구형 `calibration.json`이 있으면 보정값을 이관한다.

Listing 5. `system.json` 주요 항목

```
{
  "max_speed": 80, "collision_dist": 10, "auto_stop": 1,
  "device_name": "ARES",
  "left_calibration": 100, "right_calibration": 100,
  "enable_wheel": 1, "enable_dcmotor": 1, "enable_buzzer": 1,
  "enable_distance": 1, "enable_magsensor": 1, "enable_leds": 1,
  "enable_gun": 1, "enable_oled": 1,
  "pin_wheel_left": 13, "pin_wheel_right": 12,
  "pin_dcmotor_dir": 8, "pin_dcmotor_pwm": 9, "pin_buzzer": 15,
  "pin_distance_trig": 6, "pin_distance_echo": 7,
  "pin_magsensor": 26, "pin_gun": 22,
  "pin_leds": "21,20,19,18,17,16",
  "pin_i2c_sda": 4, "pin_i2c_scl": 5
}
```

핵심 메서드: `is_module_enabled()`, `enable_module()`, `set_pin()`(0~28 검증), `save_config()`, `save_calibration()`, `save_all()`.

4.5 펌웨어 설계상의 제약과 패턴

주요 제약

- **RAM 제한:** 대용량 버퍼 회피, 수신 버퍼 512바이트 상한, `json.indent` 미지원.
- **BLE 20바이트 체크:** 송수신 모두 20바이트 단위.
- **싱글턴 강제:** `RobotHardware` 인스턴스는 단 하나.
- **차단형 시간 명령:** `tFORWARD` 등 시간 지정 명령은 메인 루프를 점유하므로, 그 동안 다른 명령·비상정지 응답이 지연될 수 있다.

`Pico/backup/` 및 날짜 표기 백업 파일(예: `main_backup_*.py`)은 레거시·참조용으로, 수정 대상이 아니다.

주요 분석 요소 — 펌웨어의 단순함과 그 대가

설계 의도

- 단일 수신 루프 + `if/elif` 디스패치로 읽기 쉬움을 택했다.
- 부저는 비차단, 시간 명령은 차단으로 처리한 트레이드오프가 있다.
- 응답 없는 명령(fire-and-forget)으로 명령-응답 어긋남을 막는다.

분석할 때 핵심 질문

- 시간 명령(`tFORWARD, 2`)이 도는 2초 동안 비상정지가 왜 늦어질 수 있는가?
- 접두 일치 순서를 바꾸면 어떤 명령이 잘못 잡힐까?

점검 요소

- `process_data.py`에 새 명령 하나를 추가하되, 더 구체적인 접두를 먼저 두는 순서 규칙을 지켜라.

5 서비스 계층 — 웹 애플리케이션 구조

본 장은 Web/ 디렉터리의 웹 애플리케이션을 진입점, 모듈 구조, 상태·DOM²² 관리, 라우팅과 미션 로딩 측면에서 분석한다.

5.1 진입점 (HTML)

파일	역할
<code>index.html</code>	랜딩 페이지. <code>three.js</code> 로 로버 3D 모델을 렌더링하고 손 흔들기 애니메이션을 보여준 뒤 블록 편집기로 진입
<code>main.html</code>	블록 편집기 본체. Blockly 툴박스 정의, 미션 뷰, 차시 내비게이션, 대시보드를 <code>iframe</code> 으로 임베드
<code>dashboard.html</code>	관제 대시보드. 실시간 모니터링, 시스템 설정, 휠 보정, 수동 제어, 진단

5.2 ES 모듈 의존 구조

애플리케이션 컨트롤러 `main.js`가 다른 모듈을 오케스트레이션한다. 의존 관계의 핵심은 중앙 집중식 상태(`state.js`)와 상수(`constants.js`)를 여러 모듈이 공유한다는 점이다. 실제 `import` 구문을 추출한 의존 그래프는 그림 6와 같다.

²²**DOM(Document Object Model, 문서 객체 모델)**: 웹 페이지의 HTML 구조를 객체 트리로 표현한 것. 자바스크립트가 이를 통해 화면 요소를 읽고 바꾼다.

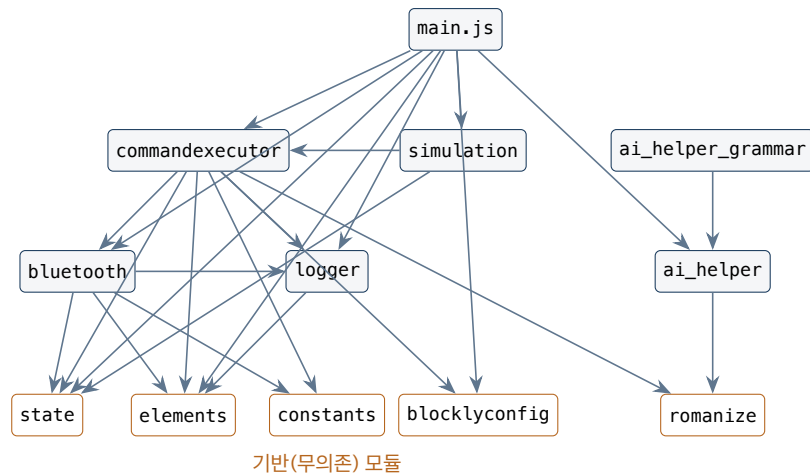


Figure 6. 웹 모듈 import 의존 연계도 (화살표 = import)

그래프에서 드러나는 특징은 다음과 같다. (1) state·elements·constants·blocklyconfig·romanize는 어떤 모듈도 import 하지 않는 기반 모듈로, 변경 파급의 시작점이다. (2) commandexecutor와 bluetooth는 가장 많은 모듈에 의존하는 허브로, 실행 경로의 중심이다. (3) simulation은 commandexecutor를 재사용하여 실기와 동일한 명령 생성 로직을 공유한다. (4) AI 파서는 ai_helper_grammar → ai_helper → romanize의 선형 의존을 가진다.

Listing 6. 모듈 의존 관계 개요

```
main.js
├── state.js      전역( 상태 객체)
├── elements.js   (DOM 참조 캐시)
├── constants.js  (UUID, 타이밍, 기본 설정)
├── blocklyconfig.js 블록( 정의틀박스/)
├── commandexecutor.js ── bluetooth.js, blocklyconfig.js,
│                       │ logger.js, constants.js, romanize.js, state.js
├── bluetooth.js  ── state.js, elements.js, logger.js, constants.js
├── simulation.js  (three.js 3D 시뮬레이터)
├── ai_helper.js / ai_helper_grammar.js
└── logger.js
```

5.3 상태 · DOM · 상수 관리

5.3.1 state.js — 전역 상태

BLE 연결 핸들(bluetoothDevice, bluetoothServer, characteristic), 알림 사용 여부, 연결 진행 플래그, 마지막 명령, 실행 상태, 사용자 변수(variables), 응답 대기 프라미스(pendingResolve/pendingReject/pendingTimeout) 등을 보관한다.

5.3.2 elements.js — DOM 참조

연결 버튼, 실행/비상정지 버튼, 저장/불러오기 버튼, 파일 입력, 로그 컨테이너 등 자주 쓰는 DOM 요소를 캐시한다.

5.3.3 constants.js — 상수

상수	값/의미
UART_SERVICE_UUID	0000ffe0-0000-1000-8000-00805f9b34fb
UART_CHARACTERISTIC_UUID	0000ffe1-0000-1000-8000-00805f9b34fb
MAX_CHUNK_SIZE	20 (바이트)
COMMAND_DELAY	100 ms (명령 간 간격)
CHUNK_DELAY	100 ms (청크 간 간격)
READ_INTERVAL	500 ms (폴링 폴백 주기)
RESPONSE_TIMEOUT	5000 ms (응답 대기 한계)
DEFAULT_SYSTEM_CONFIG	장치명, 최대속도, 충돌거리, 자동정지 등 기본값
STATUS_COLORS	연결 상태 표시 색상
STORAGE_KEYS	localStorage 키(ares-system-config)

웹 측 사용자 설정은 localStorage에 저장되어, 일부 타이밍 상수를 런타임에 사용자 정의값으로 덮어쓸 수 있다.

5.4 애플리케이션 컨트롤러 (main.js)

main.js는 다음을 담당한다.

- **차시 카탈로그**: 12개 차시 메타데이터(제목, 하드웨어, 개념, 태그)를 보유하고 개요 뷰를 구성.
- **Blockly 초기화**: 한국어 메시지로 워크스페이스 생성, 어린이용 단순화 컨텍스트 메뉴(복사/삭제/전체 삭제 등) 적용.
- **라우팅**: URL 해시 기반 내비게이션(예: #lesson=3&mission=2).
- **뷰 관리**: 개요 ↔ 차시 ↔ 미션 전환.
- **미션 로딩**: 각 차시의 lesson.json을 불러와 설명·미션 목록 구성.
- **저장/불러오기**: 미션 작업물을 Blockly XML로 직렬화.
- **예제 선택기**: 인사하기, 카운트다운 등 예제 프로그램 로딩.

5.5 대시보드 (dashboard.html)

main.html 내부에 iframe²³으로 임베드되며, 다음 기능을 제공한다.

- 연결 상태 표시(연결 시 녹색 점)
- 센서 텔레메트리(거리, 자기장)
- 수동 제어(전진/정지/좌/우, LED, 사운드)
- 시스템 설정(최대 속도 슬라이더, 충돌 거리)
- 모터 보정 및 진단

²³**iframe(inline frame)**: 한 웹 페이지 안에 다른 HTML 문서를 끼워 넣는 요소. 여기서는 대시보드를 본 화면 안에 임베드한다.

부모 창과 iframe은 `postMessage`²⁴로 상태(`STATUS, ...`)를 주고받는다.

5.6 어린이 대상 UI 설계

- 이모지 기반 블록 카테고리(🚗, ⚡, 💡, 🎧 등)로 직관성 강화.
- 평이한 한국어 블록 문구(예: “서보 전진 %1 초”).
- 어린이용 단순화 컨텍스트 메뉴(고급 옵션 제거).
- 모바일(폭 ≤ 768px)에서는 카테고리명을 이모지 위주로 축약.

주요 분석 요소 — 웹 모듈의 책임 분리

설계 의도

- ES 모듈로 책임을 나누고 `state.js`를 공유 데이터 허브로 둔다.
- 기반(무의존) 모듈과 허브 모듈을 구분해 변경 파급을 관리한다.

분석할 때 핵심 질문

- `commandexecutor.bluetooth`가 가장 많은 모듈에 의존하는 이유는?
- `state.js`를 바꾸면 영향 범위가 왜 넓은가?

점검 요소

- 의존 그래프(그림 참고)를 보고 “이 파일을 수정하면 어디가 영향받는가”를 먼저 예측한 뒤 코드로 검증하라.

6 블루투스 지원 — Web API와 파일 기반 웹앱 제작 시 요구사항

본 장은 `Web/bluetooth.js`의 `BluetoothManager`를 중심으로 Web Bluetooth 통신 구조를 분석하고, 특히 `file://` 환경에서 단독 실행되는 웹앱을 만들 때의 요구사항을 정리한다.

6.1 연결 흐름

`BluetoothManager.connect()`는 다음 순서로 BLE 연결을 수립한다.

1. **장치 탐색**: `navigator.bluetooth.requestDevice()` 호출. 이름 필터(PicoBLE, HMSoft, BT05)와 UART 서비스 필터를 병행한다.
2. **GATT 연결**: `device.gatt.connect()` 후 안정화를 위한 지연.
3. **서비스/특성 획득**: UART 서비스(`0xFFE0`)와 특성(`0xFFE1`)을 획득.
4. **알림 구독**: `startNotifications()`로 수신 알림을 구독하고, 실패 시 폴링 모드로 폴백.
5. **연결 해제 핸들러 등록**: `gattserverdisconnected` 이벤트 처리.

이 과정을 시퀀스로 보면 그림 7와 같다. 모든 단계는 사용자의 “연결” 버튼 클릭(제스처) 안에서 시작되어야 하며, 알림 구독에 실패하면 주기적 읽기(폴링)로 자동 전환된다.

²⁴**postMessage**: 서로 다른 창·iframe 사이에서 안전하게 메시지를 주고받게 하는 브라우저 API.

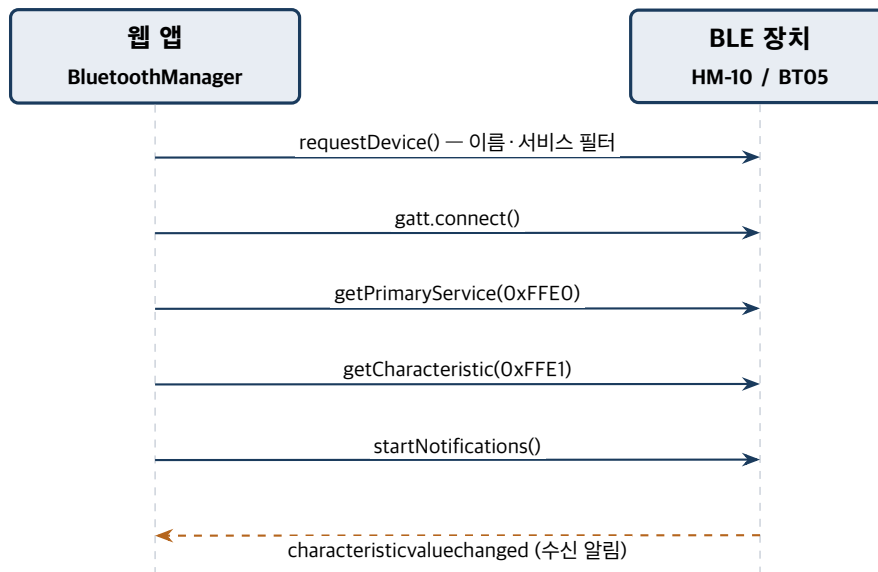


Figure 7. BLE 연결 수립 시퀀스

6.2 청크 송신

BLE 특성 쓰기는 한 번에 20바이트(MAX_CHUNK_SIZE)로 제한되므로, `sendData()`는 명령 문자열을 인코딩한 뒤 20바이트 단위로 분할 전송한다.

Listing 7. 청크 송신 핵심 로직

```

for (let i = 0; i < encoded.length; i += MAX_CHUNK_SIZE) {
  const chunk = encoded.slice(i, Math.min(i + MAX_CHUNK_SIZE, encoded.length));
  if (useWithoutResponse)
    await characteristic.writeValueWithoutResponse(chunk); // 빠름 (GATT ACK 없음)
  else
    await characteristic.writeValueWithResponse(chunk);
  // 마지막 청크 뒤에는 지연을 두지 않음 -> 단일 청크 명령 지연 제거
  if (i + MAX_CHUNK_SIZE < encoded.length)
    await this.delay(CHUNK_DELAY); // 100ms
}
  
```

청크 간 100 ms 지연은 HM-10/BT05의 연결 인터벌(약 30~70 ms)을 상회하여, 다중 청크 명령(BATCH, LED 패턴, SYS_SET)의 패킷 손실을 방지한다. 단일 청크 명령(LED_ON 등)은 마지막 청크 뒤 지연을 생략하여 응답성을 높인다. 그림 8은 긴 명령이 20바이트 청크로 분할·재조립되는 과정을 보여준다.

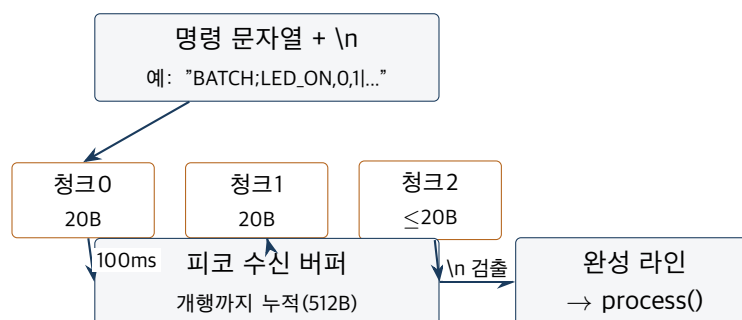


Figure 8. 20바이트 청크 분할과 펌웨어 측 개행 재조립

6.3 수신 및 응답 조립

알림 핸들러는 수신 청크를 누적 버퍼에 모으고 개행 단위로 완성된 메시지를 추출한다. 메시지 종류에 따라 분기 처리한다.

- STATUS,... — 로버 상태. iframe(대시보드)으로 postMessage 전달.
- SYS_VALUES,... — 시스템 설정 값.
- CALIB_VALUES,... — 보정 값.
- DIST:, MAG: — 센서 값. 정규식으로 추출하여 Blockly 변수에 반영.

응답 대기는 프라미스 기반이다. 빠른 피코 응답과의 경쟁 상태를 피하기 위해, 명령 전송 이전에 state.pendingResolve를 설정하고, 5초 (RESPONSE_TIMEOUT) 타임아웃과 연결 해제 시 정리(reject)를 둔다.

6.4 Web Bluetooth API 요구사항

보안 컨텍스트 및 사용자 제스처

- Web Bluetooth는 **보안 컨텍스트**에서만 동작한다: HTTPS 또는 http://localhost.
- requestDevice()는 **사용자 제스처**(클릭/터치) 안에서 호출해야 한다. ARES는 “신호 연결” 버튼 클릭으로 이를 만족한다.
- Chromium 계열(Chrome, Edge) 데스크톱에서는 file://도 보안 컨텍스트로 인정되어 Web Bluetooth가 동작한다. 모바일 Chrome은 더 엄격할 수 있어 HTTPS 호스팅을 권장한다.

6.5 파일 기반(file://) 웹앱 제작 시 요구사항

웹 소스를 그대로 file://로 열면 ES 모듈 임포트가 동작하지 않는다. <script type="module">은 file://에서 origin: null로 취급되어 동일 출처 정책²⁵에 의해 import가 차단되기 때문이다. 따라서 오프라인 단독 실행본을 만들려면 다음 요구사항을 충족해야 한다(상세 빌드 절차는 8장 참조).

1. **ES 모듈 번들링**: esbuild²⁶로 main.js와 의존 모듈을 단일 IIFE²⁷ 번들(main.bundle.js)로 묶어 import 차단을 제거한다.
2. **외부 라이브러리 로컬화**: Blockly, three.js 등을 CDN이 아닌 vendor/ 로컬 사본으로 동봉한다.
3. **스크립트 로딩 순서 보장**: type="module"을 제거하고 defer 스크립트 체인으로 전환하여 DOM 준비 후 실행 순서를 맞춘다.
4. **자산 인라인화와 fetch shim(shim)²⁸**: lesson.json, 예제 XML, GLB 모델 등 fetch로 로드하던 자산을 base64/텍스트로 인라인하고, window.fetch를 가로채는 shim을 설치해 네트워크 없이 자산을 공급한다.

이러한 처리를 통해, 인터넷이 없는 교실 환경에서도 USB나 내부망으로 배포된 단일 폴더를 브라우저로 더블클릭하여 블록 코딩과 BLE 로버 제어를 모두 수행할 수 있다.

²⁵**동일 출처 정책(Same-Origin Policy)**: 한 출처(origin)의 문서·스크립트가 다른 출처의 리소스에 접근하는 것을 제한하는 브라우저 보안 규칙. file://는 출처가 null이 되어 모듈 import가 막힌다.

²⁶**esbuild**: 자바스크립트·CSS를 매우 빠르게 하나로 묶어 주는 번들러(빌드 도구).

²⁷**IIFE(Immediately Invoked Function Expression, 즉시 실행 함수 표현식)**: 정의와 동시에 한 번 실행되는 함수. 내부 변수를 외부와 격리(캡슐화)하는 데 쓴다.

²⁸**shim(shim)**: 기존 함수·API 호출을 가로채 다른 동작으로 대체하거나 보완하는 보정 코드. 여기서는 fetch를 가로채 인라인된 자산을 돌려준다.

주요 분석 요소 — 무선 통신의 제약과 우회

설계 의도

- BLE 20바이트·연결 인터벌 한계에 맞춰 청크 사이에 지연을 둔다.
- 응답을 받기 전에 프라미스를 먼저 등록해, 빠른 응답과의 경쟁 상태를 피한다.
- file:// 제약은 빌드(번들 + 심)로 우회한다.

분석할 때 핵심 질문

- 청크 지연을 0으로 하면 긴 명령에서 무엇이 깨질까?
- 왜 단일 청크 명령은 마지막 지연을 생략하는가?

점검 요소

- CHUNK_DELAY 등 타이밍 상수를 바꿔 가며 통신 안정성과 반응 속도의 변화를 관찰하라.

7 코드 블록 구현 방식 — 웹앱 제작 시 요구사항

본 장은 Google Blockly 기반 블록 코딩이 어떻게 정의되고, 어떻게 실행 가능한 명령으로 변환·전송되는지를 blocklyconfig.js와 commandexecutor.js 중심으로 분석한다.

7.1 블록 정의 (blocklyconfig.js)

커스텀 블록은 Blockly의 JSON 스키마로 정의된다. 각 블록은 고유 type, 시각적 레이블 message0, 입력/필드 정의 args0, 카테고리 색상 colour, 한국어 도움말 tooltip을 가진다.

Listing 8. 블록 정의 예시(개념)

```
{
  "type": "timed_forward",
  "message0": "🚗 서보 전진 %1 초",
  "args0": [{ "type": "input_value", "name": "SECONDS", "check": "Number" }],
  "previousStatement": null, "nextStatement": null,
  "colour": "#FF8C00",
  "tooltip": "지정한 시간 동안 앞으로 이동합니다"
}
```

7.2 블록 카테고리(툴박스)

툴박스는 main.html의 XML로 정의되며, 이모지 라벨을 가진 카테고리들로 구성된다.

카테고리	색상	대표 블록
🚗 서보 모터	#FF8C00	timed_forward/backward/left/right, move_forward, turn_left/right, stop_moving
⚡ DC 모터	#FFCC00	main_motor_forward_timed, main_motor_forward, main_motor_stop

카테고리	색상	대표 블록
💡 LED	#FF5555	led_on, led_off, led_off_all, set_lamp
🖥️ 디스플레이	#9966FF	send_message, send_message_xy, display_icon, clear_display, clear_rect
🔊 소리	#00CCFF	buzzer_on, buzzer_note(3옥타브 음계)
🔫 발사	#FF4500	gun_fire
📡 센서	#5C81A6	pico_check_device, check_distance, check_magnetic
🕒 시간	#5CA65C	time_sleep
🔧 제어	(내장)	controls_repeat_ext, controls_whileUntil, controls_if
🚀 한꺼번에	#8E44AD	batch_block
📦 변수 / 📐 수학 · 논리 / 🔑 함수	(내장)	Blockly 표준 블록

7.3 블록 → 명령 변환 (commandexecutor.js)

CommandExecutor는 블록 트리를 순회하며 각 블록을 프로토콜 명령 문자열로 변환하고 전송한다.

7.3.1 명령 생성: generateCommand(block)

블록 타입	생성 명령
timed_forward	SERVO_tFORWARD, \${초}
move_forward	SERVO_FORWARD
main_motor_forward_timed	DC_tFORWARD, \${초}
led_on	LED_ON, \${번호}, \${밝기}
led_off_all	LED_OFF, ALL
set_lamp	[\${v0} \${v1} ... \${v5}]
send_message	MSG, \${로마자변환텍스트}
display_icon	ICON, \${이름}, \${x}, \${y}
buzzer_on / buzzer_note	BUZZER_ON, \${주파수}, \${지속}
gun_fire	GUN_FIRE
check_distance	DISTANCE (→ 변수 _last_distance)
check_magnetic	MAGNET (→ 변수 _last_magnetic)
pico_check_device	PING

블록 타입	생성 명령
time_sleep	SLEEP, \${초}

OLED는 ASCII 폰트만 지원하므로, send_message의 한글 텍스트는 romanize.js로 로마자 변환된 뒤 전송된다(예: “안녕” → “annyeong”).

7.3.2 값 블록 평가: evaluateValueBlock(block)

숫자·텍스트·변수·산술·비교·논리·난수·함수 반환 블록을 재귀적으로 평가하여 문자열 값을 산출한다(0으로 나눗셈 보호 포함). 센서 값은 state.variables의 _last_distance, _last_magnetic에 저장되어 조건식에서 참조된다.

7.3.3 블록 처리와 제어흐름: processBlock(block)

블록 체인을 순회하며 반복(controls_repeat_ext), 조건(controls_if), while/until(controls_whileUntil), 변수 설정을 해석한다. 무한 루프 방지를 위해 반복 횟수와 중첩 깊이에 상한을 둔다.

블록 하나가 실행 경로로 들어가 명령으로 변환·전송되기까지의 파이프라인을 정리하면 그림 9와 같다. 블록은 성격에 따라 세 갈래로 분기한다. (1) batch_block은 평탄화 후 일괄 전송, (2) 제어흐름·변수·함수 블록은 handleLogicBlock이 재귀 처리, (3) 그 외 동작 블록은 generateCommand로 명령 문자열을 만들어 전송한다. 조건식·인자는 evaluateValueBlock이 재귀 평가하여 값을 공급한다.

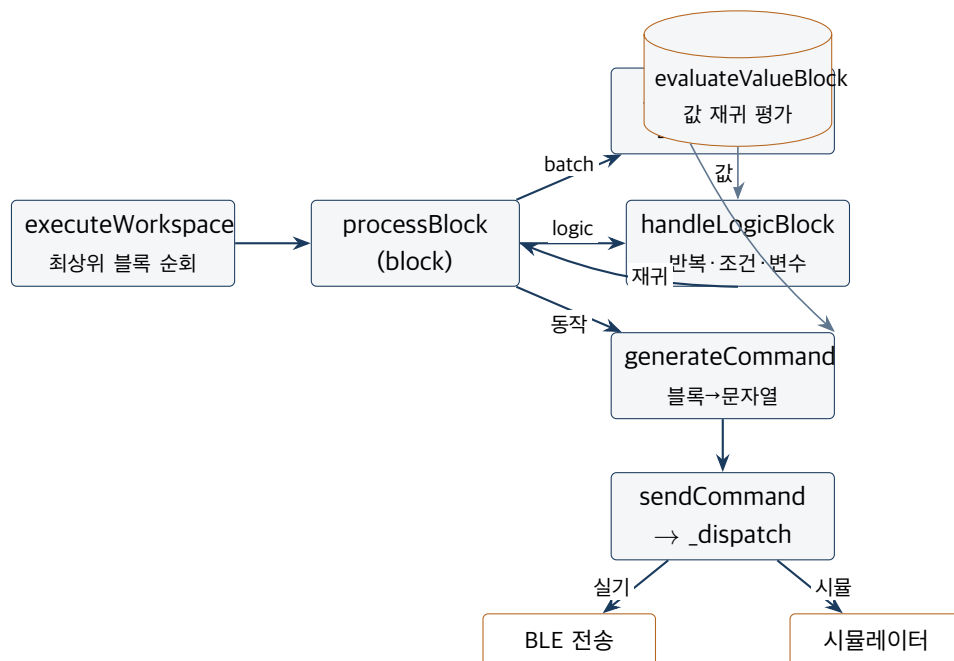


Figure 9. 블록 → 명령 처리 파이프라인

7.3.4 명령 디스패치와 Fire-and-forget

sendCommand()는 명령 헤드가 FIRE_AND_FORGET_HEADS 집합에 속하는지로 응답 대기 여부를 결정한다. 시뮬레이션 모드에서는 BLE 대신 시뮬레이터 싱크(simSink)로 명령을 보낸다.

- **응답 불필요(fire-and-forget):** LED_*, MSG*, ICON, SERVO_*/DC_* 연속 구동, BUZZER_ON, SYS_SET 등.

- **응답 대기:** DISTANCE, MAGNET, PING, SLEEP, tFORWARD 계열(시간 명령), BATCH.

7.4 한꺼번에 실행: batch_block

batch_block은 내부 블록들을 평탄화하여 BATCH;cmd1|cmd2|... 단일 페이로드로 전송하고, 피코로부터 단일 ACK를 받는다. 센서·제어흐름·변수·함수 블록은 배치 내부에 넣을 수 없으며(BATCH_FORBIDDEN_TYPES), onchange 검증기가 위반 시 경고한다.

7.5 웹앱 제작 시 요구사항

새 블록 추가 시 변경 지점

새로운 블록을 추가하려면 일반적으로 세 곳을 함께 수정해야 한다.

1. blocklyconfig.js — 블록 정의(타입, 메시지, 인자, 색상).
2. main.html — 툴박스 XML에 블록 등록(필요 시 shadow 기본값).
3. commandexecutor.js — 블록 타입 → 명령 문자열 생성 및 응답 정책.

추가로, 새 명령이 펌웨어 동작을 요구한다면 Pico/process_data.py의 CommandProcessor에 대응 핸들러를 추가해야 한다(9장 연계 구조 참조). OLED로 출력되는 한글 텍스트는 로마자 변환 경로를 거치도록 하고, 다중 청크가 되는 긴 명령은 BLE 청크 타이밍을 고려해야 한다.

주요 분석 요소 — 블록에서 명령으로

설계 의도

- 블록을 batch/제어흐름/동작 세 갈래로 나눠 처리한다.
- 명령 생성 로직을 하나로 두어, 시뮬레이터와 실기가 같은 의미로 동작한다.

분석할 때 핵심 질문

- 새 블록을 추가할 때 왜 blocklyconfig.js·툴박스·commandexecutor.js 세 곳을 함께 고쳐야 하는가?
- 시뮬과 실기가 같은 명령을 쓰면 교육적으로 무엇이 좋은가?

점검 요소

- 블록 1개를 정의 → 툴박스 등록 → 명령 생성까지 끝까지 추가하라(펌웨어 동작이 필요하면 핸들러도 함께).

8 WebGL 기반 디지털 플랫폼 기술 — 서비스 구조 및 제작 요구사항

본 장은 three.js 기반 WebGL 3D 렌더링이 ARES에서 어떻게 활용되는지를 랜딩 페이지, 미션 시뮬레이터, 오프라인 빌드 측면에서 분석하고, 웹앱 기반 제작 시 요구사항을 정리한다.

8.1 three.js 번들링 전략

three.js는 CDN이 아닌 로컬 사전 번들로 동봉된다.

- Web/vendor/three-bundle.min.js — window.THREE 네임스페이스와, GLTFLoader·OrbitControls·RoomEnvironment를 담은 window.ARES3 네임스페이스를 노출한다.
- Web/vendor/meshopt_decoder.js — GLB 모델이 EXT_meshopt_compression으로 압축되어 있어, GLTFLoader가 파싱하기 전에 반드시 먼저 로드되어야 한다.

로컬 번들 채택 이유는 file:/// 오프라인 환경 지원이다. CDN 페치 의존을 제거함으로써 인터넷 없는 교실에서도 3D 렌더링이 동작한다.

8.2 랜딩 페이지 3D (index.html)

랜딩 페이지는 로버 모델을 인터랙티브하게 렌더링한다.

- **렌더러**: WebGLRenderer(antialias, alpha), sRGB 출력, ACES 필름릭 톤 매핑, 소프트 새도우(PCFSoftShadowMap).
- **환경광**: RoomEnvironment 기반 PMREM 환경맵 + 3점 조명 (반구광·키 라이트·필 라이트).
- **카메라/컨트롤**: PerspectiveCamera(약 40° FOV), OrbitControls로 좌우 회전 허용, 상하 틸트 $\pm 10^\circ$ 제한, 줌·팬 비활성.
- **본 애니메이션**: 모델의 팔 본(RightArm, RightForeArm)을 쿼터니언 보간으로 움직여 “손 흔들기” 인사를 구현.

8.2.1 file:///에서의 모델 로딩

랜딩 로버 모델은 Mesh/ares_robot.glb와, 이를 base64로 임베드한 Mesh/ares_robot.embed.js (window.ARES_ROBOT_GLB_B64)로 제공된다. 프로토콜에 따라 분기한다.

Listing 9. file:// 분기 로딩

```
if (location.protocol === 'file:') {
  ensureEmbed() // base64 임베드 스크립트 로드
  .then(b64 => loader.parse(b64ToBuffer(b64), '', onModelLoad, onModelErr));
} else {
  loader.load('Mesh/ares_robot.glb', onModelLoad, undefined, onModelErr);
}
```

file:///에서는 fetch 대신 base64 ArrayBuffer를 직접 GLTFLoader.parse()에 넘겨 오프라인 로딩을 보장한다.

8.3 미션 시뮬레이터 (simulation.js)

simulation.js는 실제 BLE 연결 없이 블록 프로그램을 3D로 시연하는 시뮬레이터이다. 주제(TOPICS)별로 다른 환경 모델과 연출을 로드한다.

주제	모델	연출
알비	Mesh/AlbiStaticLow.glb	눈 LED 구체·점광, OLED 캔버스 텍스처
신호등	Mesh/LampBox.glb 외	신호등 램프·손 모델

주제	모델	연출
발사대	Mesh/LaunchStation.glb	안테나/로켓 메시 분리 재질, 레이더, 발사 LED 스트립
로버	Mesh/RoverParts/*.glb	7개 부품(본체·건·헤드·LED·OLED·레이더·휠) 조립

시뮬레이터는 블록 실행 시 명령을 UI 로그로 출력하고, 눈 LED·OLED 텍스처를 실시간 갱신 하며, 발사대 주제에서는 지오메트리 인덱스를 분석해 안테나/로켓을 별도 메시로 분리하는 후처리 (recolorLaunchpadAntenna)를 수행한다.

8.4 오프라인 단독 실행본 빌드

build.ps1(Windows) / build.sh(macOS·Linux) / build.bat(진입점)는 Web/을 file://로 동작하는 단일 Build/ 폴더로 변환한다. 주요 단계는 다음과 같다.

1. Build/ 초기화.
2. **ES 모듈 번들링**: esbuild로 main.js와 의존 모듈을 단일 IIFE main.bundle.js로 묶음.
3. **외부 라이브러리 로컬화**: Blockly 압축본을 vendor/로 내려받음.
4. **정적 자산 복사**: 대시보드, CSS, three.js 번들, meshopt 디코더, 임베드 로봇 등.
5. **index.html·main.html 패치**: type="module" 제거, defer 스크립트 체인으로 전환, 경로 평탄화.
6. **자산 인라인화**:
 - GLB 바이너리는 모델별 vendor/bin_<이름>.js로 분할 (base64; GitHub 100 MB 파일 제한 회피).
 - 텍스트 자산(overview.html, lesson.json, 예제 XML)은 vendor/inline_assets.js의 DATA 에 등록.
 - window.fetch를 가로채는 심을 설치하여, Mesh/*.glb· 텍스트 요청을 인라인 데이터로 응답.

Listing 10. inline_assets.js의 fetch 심(개념)

```

window.fetch = function (input, init) {
  const url = norm(input);
  const bk = find(BIN, url); // 바이너리(GLB) 우선 매칭
  if (bk !== null)
    return Promise.resolve(new Response(b64ToU8(BIN[bk]),
      { status: 200, headers: { 'Content-Type': 'application/octet-stream' } }));
  const tk = find(DATA, url); // 텍스트 자산
  if (tk !== null)
    return Promise.resolve(new Response(DATA[tk],
      { status: 200, headers: { 'Content-Type': mimeFor(tk) } }));
  return origFetch ? origFetch(input, init) : Promise.reject(...);
};

```

defer 로딩 순서는 (1) bin_*.js가 window.__ARES_BIN__을 채우고, (2) inline_assets.js가 fetch 심을 설치하며, (3) main.bundle.js가 앱을 시작하도록 보장된다. 이 순서와 런타임 자산 가로채기의 연계는

그림 10와 같다. 핵심은 앱이 평소처럼 `fetch('Mesh/...glb')`를 호출하더라도, 설치된 심이 이를 가로채 인라인된 base64를 Response로 돌려주므로 앱 코드를 수정하지 않고도 오프라인 동작이 가능하다는 점이다.

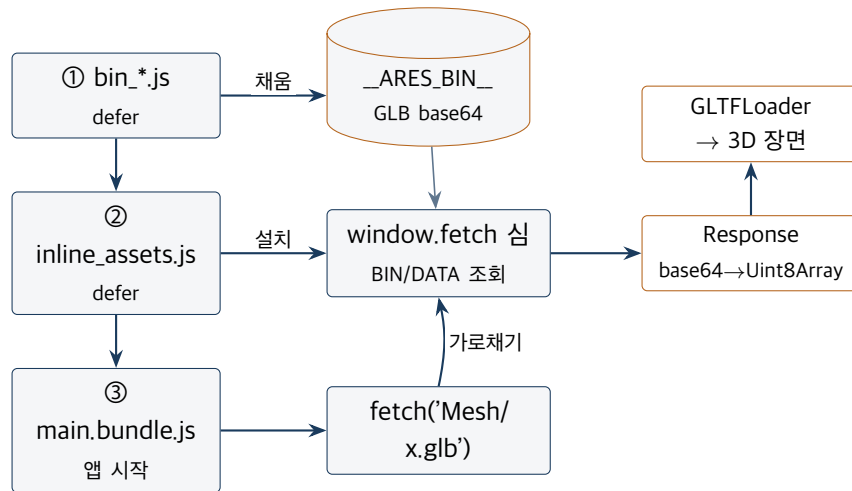


Figure 10. 오프라인 빌드 자산 로딩 · fetch 심 연계도

8.5 웹앱 기반 제작 시 요구사항

WebGL 자산 제작 · 배포 체크리스트

- GLB 모델은 EXT_meshopt_compression으로 압축하고, 디코더를 GLTFLoader보다 먼저 로드한다.
- file:// 지원을 위해 모든 모델 로딩에 fetch 또는 parse 경로를 일관되게 두고, 오프라인 빌드에서 fetch 심으로 가로챌 수 있게 한다.
- 대용량 GLB는 모델별 base64 청크로 분할하여 단일 파일 크기 제한을 피한다.
- three.js·Blockly 등은 로컬 사본으로 동봉하여 CDN 비의존성을 확보한다.
- 스크립트 로딩 순서를 defer 체인으로 명시하여 전역 네임스페이스 준비 순서를 보장한다.

주요 분석 요소 — 오프라인으로 만드는 법

설계 의도

- three.js·Blockly를 로컬 동봉하고, fetch 심으로 자산을 인라인해 file:// 단독 실행을 완성한다.
- defer 로딩 순서가 동작의 전제다.

분석할 때 핵심 질문

- fetch 심이 없으면 file://에서 무엇이 실패하는가?
- 왜 GLB를 모델별 base64 청크로 나눴는가?

점검 요소

- 빌드 스크립트(build.sh/build.ps1)를 직접 실행해 Build/를 만들고, 브라우저로 더블클릭해 동작을 확인하라.

9 코드 별 연계 구조

본 장은 앞 장들에서 개별 분석한 구성요소들이 실제 동작 시 어떻게 맞물리는지를 종단 간(end-to-end) 연계 관점에서 정리한다.

9.1 종단 간 명령 처리 시퀀스

블록 하나가 실행되어 하드웨어가 반응하기까지의 호출 사슬은 다음과 같다.

Listing 11. 블록 1개 실행의 호출 사슬

```

웹
[]
main.js : 실행 버튼
-> commandexecutor.processBlock(block)
-> generateCommand(block)    // "SERVO_tFORWARD,2"
-> sendCommand(cmd)         // fire-and-forget 여부 판단
-> bluetooth.sendData(cmd, wait)
    -> 바이트20 청크 write (0xFFE1 특성)
[BLE 모듈] -> UART 9600피코
[]
main.py : _read_uart_line()    // 개행까지 버퍼 조립
-> CommandProcessor.process("SERVO_tFORWARD,2")
-> _handle_timed_wheel()
    -> robot.wheel.forward(speed) // 초2 차단 후 정지
-> 응답 필요 시 _send_response("1") // 바이트20 청크웹
[]
bluetooth.handleRxData() -> pendingResolve("1") // 프라미스 해소

```

같은 사슬을 시간 축 시퀀스로 표현하면 그림 11와 같다. 6개 참여 요소가 호출(실선)과 응답(점선)으로 이어지며, 시간 명령(tFORWARD,2)의 경우 wheel.forward가 2초간 펌웨어를 점유한 뒤에야 응답이 역방향으로 전파되는 것을 알 수 있다.

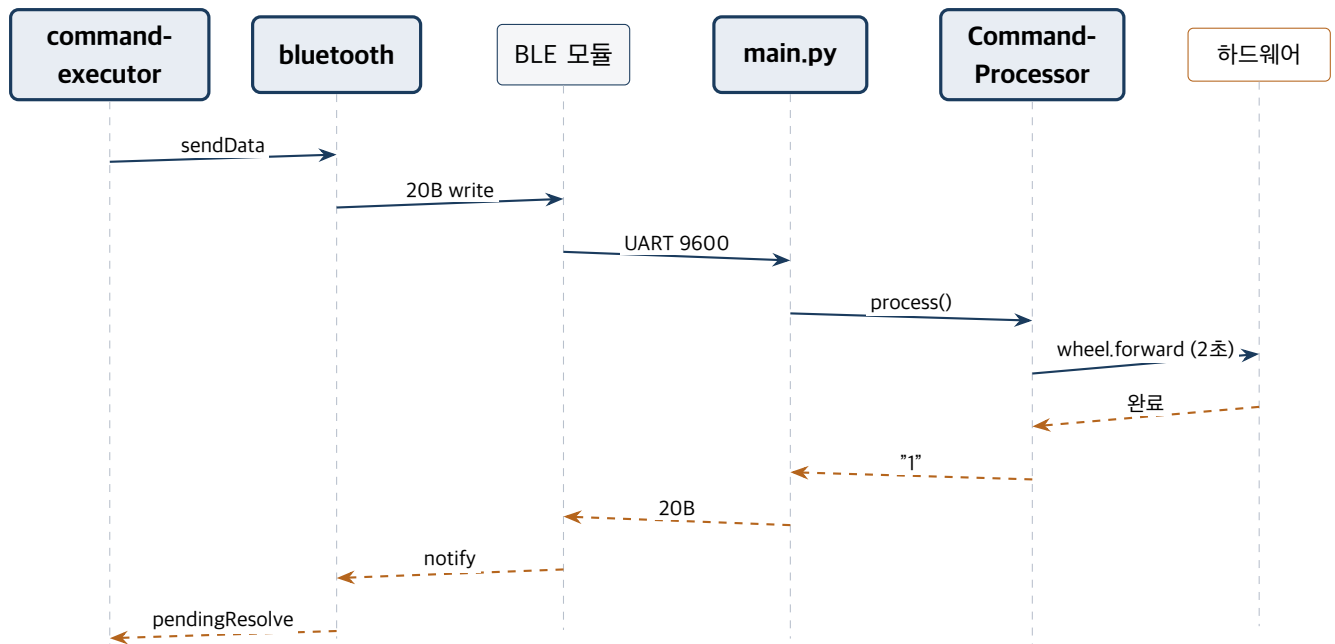


Figure 11. 종단 간 명령·응답 시퀀스 연계도

9.2 모듈 간 결합 지점

결합 지점	웹 측	펌웨어 측
명령 프로토콜 문자열	commandexecutor.js	process_data.py
BLE UUID/청크 규약	constants.js, bluetooth.js	main.py(UART)
센서 변수 반영	bluetooth.js → state.variables	DIST:/MAG: 응답
시스템 설정 동기화	dashboard.html	GET_SYS/SYS_SET
모듈 활성 상태	대시보드 토크	system.json/SET_MODULE

9.3 명령 프로토콜: 두 코드베이스의 계약

웹과 펌웨어는 코드를 공유하지 않으므로, 명령 문자열 프로토콜이 둘 사이의 유일한 계약(contract)이다. 따라서 한쪽의 명령 형식 변경은 반드시 다른 쪽과 함께 갱신되어야 한다.

프로토콜 변경 시 동기화 규칙

- 웹에서 새 명령을 생성하면(commandexecutor.js), 펌웨어 핸들러(process_data.py)를 함께 추가한다.
- 응답을 기대하는 명령은 양측의 fire-and-forget 목록 (FIRE_AND_FORGET_HEADS ↔ NO_RESPONSE_CMDS)을 일치시켜 명령-응답 오정렬을 방지한다.
- 인자 구분자(침표)·종단(개행)·20바이트 체크 규약을 양측이 동일하게 따른다.
- 프로토콜 명세는 Document/API_DOCUMENTATION.md를 단일 출처로 유지한다.

9.4 상태 공유와 데이터 결합 (웹 내부)

웹 클라이언트 내부에서는 state.js가 사실상의 공유 데이터 허브이다. bluetooth.js가 수신한 센서 값을 state.variables에 기록하면, commandexecutor.js의 조건식 평가가 이를 읽어 제어 흐름을 결정한다. 대시보드(iframe)와 본체는 postMessage로 STATUS 텔레메트리를 주고받아 상태를 동기화한다.

9.5 설정의 이중 소유와 정합성

시스템 설정은 두 곳에 존재한다. 펌웨어의 system.json(정본)과 웹의 localStorage(ares-system-config, 캐시·UI 기본값)이다. 대시보드는 GET_SYS로 펌웨어 값을 읽어 표시하고, SYS_SET/ CALIB_SET으로 펌웨어에 반영한다. 따라서 펌웨어 값이 실효(즉 실제 적용되는) 기준이며, 웹은 이를 조회·갱신하는 클라이언트로 동작한다(그림 12).

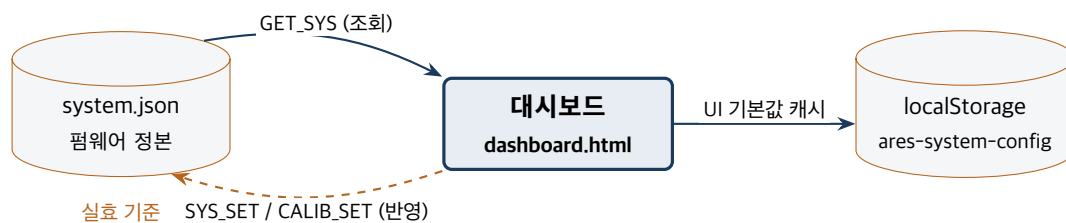


Figure 12. 설정의 이중 소유와 동기화 연계도

9.6 시뮬레이션과 실기의 분기

commandexecutor.js의 디스패치는 시뮬레이션 여부에 따라 갈린다. 시뮬레이션 모드에서는 동일한 명령 문자열이 BLE 대신 simulation.js의 싱크로 전달되어 3D로 시연된다. 즉, 명령 생성 로직은 단일이고 전송 대상만 교체되므로, 시뮬레이터와 실기의 동작 의미가 일치한다. 이는 “동일 블록 → 동일 명령 → (시물 또는 실기)” 구조로, 교육 현장에서 로버 없이도 동일한 코딩 경험을 제공하는 핵심 설계이다.

주요 분석 요소 — 두 코드베이스를 잇는 계약

설계 의도

- 웹과 펌웨어의 유일한 계약은 명령 문자열이다. 한쪽을 바꾸면 반드시 다른 쪽을 함께 바꿔야 한다.
- 시물/실기 분기는 전송 대상만 바꾸고 명령 의미는 유지한다.

분석할 때 핵심 질문

- fire-and-forget 목록이 양측에서 어긋나면 어떤 증상이 나타날까?

- 설정이 system.json과 localStorage 두 곳에 있을 때 무엇이 “정답”인가?

점검 요소

- 명령 하나의 형식을 바꿔 보고, 웹·펌웨어 양쪽에서 수정해야 할 지점을 빠짐없이 목록화하라.

10 네이티브 앱 (macOS, iOS, Android, Desktop) 제작 방식

ARES는 현재 웹 단독(HTML5 + Web Bluetooth + WebGL) 아키텍처로, 별도의 네이티브 앱 프레임워크 (Electron, Capacitor 등)를 사용하지 않는다. 본 장은 기존 웹 자산을 최대한 재사용하면서 각 플랫폼용 네이티브 앱으로 패키징하는 방안을 제시한다.

10.1 핵심 제약: 플랫폼별 BLE 지원 차이

네이티브 패키징의 성패는 **Web Bluetooth 지원 여부**에 달려 있다. 현재 통신 계층(bluetooth.js)은 navigator.bluetooth에 직접 의존하는데, 이 API의 지원은 플랫폼마다 크게 다르다.

실행 환경	Web Bluetooth	합의
데스크톱 Chrome/Edge	지원	그대로 동작(현행 방식)
Electron(데스크톱)	지원(핸들러 설정)	웹 코드 거의 그대로 재사용
Android Chrome	지원	브라우저로는 동작
Android WebView	미지원	래퍼 앱은 네이티브 BLE 브리지 필요
iOS Safari/WKWebView	미지원	네이티브 BLE 브리지 필수

설계 권고: 통신 계층 추상화

네이티브화의 선행 작업으로, bluetooth.js를 전송(transport) 인터페이스로 추상화할 것을 권한다. 즉 connect(), sendData(), onReceive() 등을 인터페이스로 정의하고, 구현체를 (1) Web Bluetooth, (2) 네이티브 BLE 플러그인 브리지로 교체 가능하게 만든다. 명령 생성 (commandexecutor.js)과 UI는 전송 구현에 무관하게 유지된다. 이 추상화 한 번으로 모든 플랫폼 분기를 흡수할 수 있다.

권고하는 전송 추상화의 연계 구조는 그림 13와 같다. 상위 모듈(명령 생성·UI)은 전송 인터페이스에만 의존하고, 실제 구현체는 플랫폼에 따라 교체된다. 현행 Web Bluetooth 구현은 데스크톱·Android 브라우저·Electron에서 그대로 쓰이고, WKWebView·Android WebView처럼 Web Bluetooth가 없는 환경에서는 네이티브 BLE 플러그인 구현으로 대체한다.

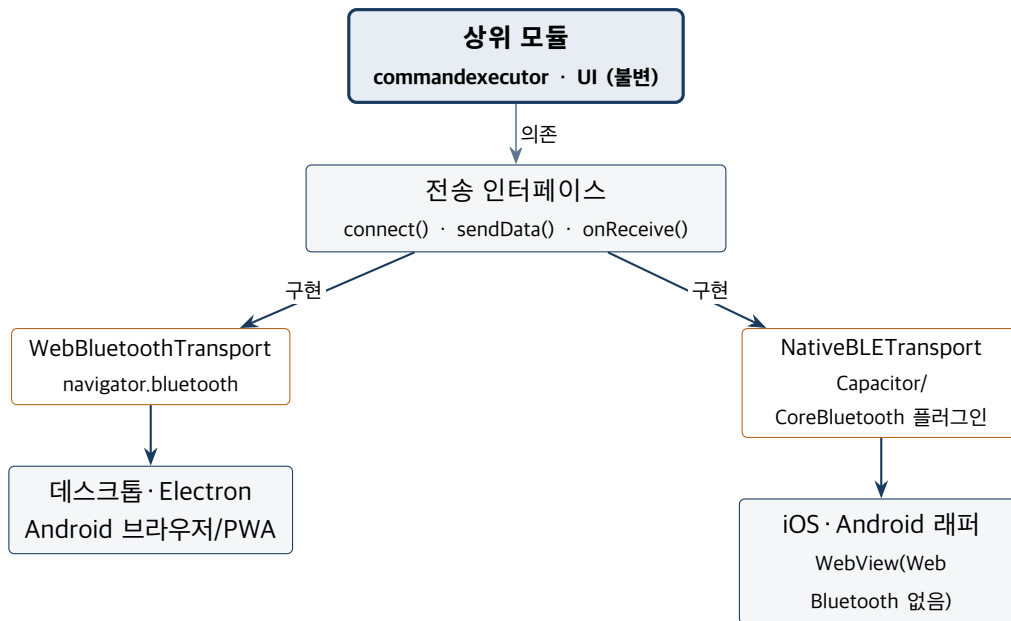


Figure 13. 전송 계층 추상화 · 플랫폼별 구현 연계도

10.2 데스크톱 (macOS / Windows / Linux)

10.2.1 방안 A — Electron (권장)

Electron²⁹은 Chromium³⁰을 내장하므로 Web Bluetooth를 지원하며, 기존 Build/ 산출물을 거의 수정 없이 탑재할 수 있다.

- 메인 프로세스에서 select-bluetooth-device 이벤트를 처리하여 장치 선택 UI를 연결한다(Electron은 기본 브라우저식 장치 선택기를 제공하지 않으므로 핸들러가 필요).
- 오프라인 빌드의 fetch 심 · 로컬 vendor 자산을 그대로 활용한다.
- electron-builder로 macOS .dmg/.app, Windows .exe/installer, Linux AppImage 생성.
- macOS는 BLE 사용을 위해 NSBluetoothAlwaysUsageDescription 권한 고지와 코드 서명 · 공증(notarization)이 필요하다.

10.2.2 방안 B — Tauri (경량 대안)

Tauri³¹는 시스템 WebView + Rust 백엔드로 번들 크기가 작다. 다만 macOS의 WKWebView³²는 Web Bluetooth를 지원하지 않으므로, Rust 측 BLE 라이브러리 (예: btbleplug)로 네이티브 BLE를 구현하고 JS와 브리지해야 한다. 위의 전송 추상화가 전제된다.

10.3 Android

- **Capacitor³³ 래퍼**: Build/ 웹 자산을 Capacitor 프로젝트의 웹 디렉터리로 사용한다. 단, Android System WebView는 Web Bluetooth를 지원하지 않으므로, BLE는 네이티브 플러그인 (예:

²⁹**Electron**: 웹 기술(HTML/CSS/JS)로 데스크톱 앱을 만드는 프레임워크. Chromium 브라우저 엔진과 Node.js를 내장한다.

³⁰**Chromium**: Google Chrome · Microsoft Edge 등의 기반이 되는 오픈소스 브라우저 엔진.

³¹**Tauri**: 시스템에 내장된 WebView와 Rust 백엔드를 사용해 가벼운 데스크톱 앱을 만드는 프레임워크.

³²**WKWebView**: iOS · macOS 앱에 웹 콘텐츠를 표시하는 애플의 웹뷰 컴포넌트. Web Bluetooth를 지원하지 않는다.

³³**Capacitor**: 웹 앱을 iOS · Android 네이티브 앱으로 감싸고, 카메라 · 블루투스 같은 네이티브 기능을 플러그인으로 연결해 주는 런타임.

@capacitor-community/bluetooth-le)으로 처리하고, 전송 구현체를 이 플러그인 호출로 교체한다.

- **권한**: BLUETOOTH_SCAN, BLUETOOTH_CONNECT (Android 12+), 구형 기기 스캔을 위한 위치 권한을 매니페스트에 선언한다.
- **대안(PWA³⁴)**: Android Chrome은 Web Bluetooth를 지원하므로, 설치형 PWA로 배포하면 네이티브 빌드 없이 홈 화면 설치·오프라인 캐시가 가능하다.

10.4 iOS

iOS는 Safari·WKWebView 모두 Web Bluetooth를 지원하지 않는다. 따라서 순수 웹 방식으로는 BLE 제어가 불가능하며, 다음 중 하나가 필요하다.

- **Capacitor + 네이티브 BLE 플러그인**: 웹 UI는 재사용하되, BLE는 CoreBluetooth 기반 플러그인 (@capacitor-community/bluetooth-le)으로 구현하고 전송 추상화로 연결한다. App Store 배포 시 NSBluetoothAlwaysUsageDescription 고지가 필수이다.
- **Web Bluetooth 지원 브라우저(WebBLE/Bluefy)**: 별도 앱 없이 해당 서드파티 브라우저로 웹앱을 여는 임시 방안. 배포·통제 측면에서 교육 현장 권장도는 낮다.

10.5 플랫폼별 권장 조합 요약

플랫폼	권장 방식	BLE 처리
macOS/Win/Linux	Electron	Web Bluetooth(핸들러 설정) — 코드 재사용 최대
Android	PWA 또는 Capacitor	PWA는 Web Bluetooth, 래퍼는 네이티브 BLE 플러그인
iOS	Capacitor	네이티브 BLE 플러그인(CoreBluetooth) 필수
공통 선행작업	전송 추상화	bluetooth.js를 인터페이스화

10.6 단계적 도입 로드맵

1. bluetooth.js를 전송 인터페이스로 리팩터링(Web Bluetooth 구현을 기본 구현체로 유지).
2. Electron 데스크톱 앱부터 출시(코드 재사용도가 가장 높음).
3. Android는 PWA로 우선 제공, 필요 시 Capacitor + BLE 플러그인으로 승격.
4. iOS는 Capacitor + 네이티브 BLE 플러그인으로 구현(가장 큰 추가 작업).
5. 코드 서명·공증·스토어 심사·권한 고지 등 배포 요건을 플랫폼별로 정리.

주요 분석 요소 — 한 번에 여러 플랫폼으로

설계 의도

- 플랫폼마다 다른 BLE 지원 차이를 전송 추상화 한 곳에서 흡수해, 상위 코드를 건드리지 않는다.

³⁴**PWA(Progressive Web App)**: 브라우저로 접속하지만 홈 화면 설치·오프라인 사용·푸시 알림 등 네이티브 앱에 가까운 경험을 제공하는 웹 앱.

분석할 때 핵심 질문

- 왜 iOS는 웹만으로 BLE 제어가 안 되는가?
- 전송 추상화를 먼저 해 두면 네이티브 확장이 왜 쉬워지는가?

점검 요소

- bluetooth.js를 connect/sendData/onReceive 인터페이스로 분리하는 설계를 종이에 그려 보라.

11 주요 기술 요소 및 개선 계획

본 장은 ARES의 핵심 기술 요소를 요약하고, 코드 분석에서 관찰된 사항을 바탕으로 한 개선 계획을 제시한다.

11.1 핵심 기술 요소 요약

기술 요소	내용
시각적 블록 코딩	Google Blockly 기반 커스텀 블록 + 한국어 UI + 어린이용 단순화
텍스트 명령 프로토콜	개행 종단·심표 구분의 단순 텍스트 계약(웹↔펌웨어)
Web Bluetooth 통신	GATT UART(0xFFE0/0xFFE1), 20바이트 청크, 응답 대기/Fire-and-forget
모듈식 펌웨어	싱글턴 하드웨어 추상화 + system.json 기반 모듈·핀 런타임 구성
WebGL 시뮬레이터	three.js로 실기 없이 동일 명령을 3D 시연
오프라인 단독 실행	esbuild 번들 + 자산 인라인 + fetch 심으로 file:// 지원
자연어 보조	문법 기반 NL→블록 파서 + 로컬 LLM PoC

11.2 개선 계획

11.2.1 1. 통신 계층 추상화

현재 bluetooth.js는 navigator.bluetooth에 직접 결합되어 있다. 이를 전송 인터페이스로 추상화하면 (1) 네이티브 앱(10장)으로의 확장, (2) 시뮬레이션/실기 전환 일원화, (3) 단위 테스트용 목(mock) 전송 주입이 용이해진다. 명령 생성과 UI는 변경 없이 유지된다.

11.2.2 2. 명령 프로토콜의 단일 출처화

웹과 펌웨어가 명령 문자열만으로 결합되어 있어, 한쪽 변경 시 다른 쪽 누락이 회귀(regression)를 유발할 수 있다. 명령 명세를 기계가 읽을 수 있는 단일 정의(예: 명령·인자·응답 여부를 담은 표)에서 양측 코드 일부를 생성하거나 검증하도록 하면 정합성이 강화된다. 최소한 FIRE_AND_FORGET_HEADS와 NO_RESPONSE_CMDS의 일치를 자동 점검하는 절차를 둔다. 목표 구조는 그림 14와 같다. 현재는 두 코드베이스가 명령 형식을 각자 보유(점선)하지만, 단일 명세에서 양측 일부를 생성·검증(실선)하도록 바꾸면 회귀를 구조적으로 차단할 수 있다.

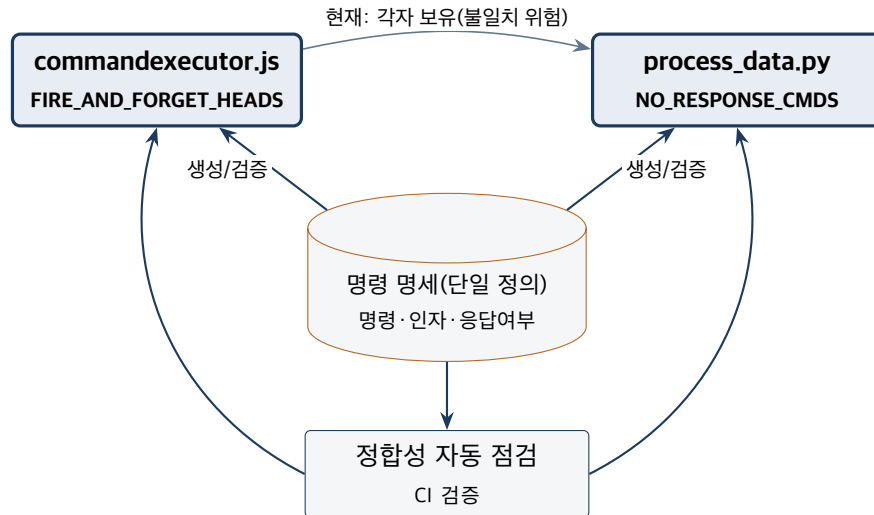


Figure 14. 명령 프로토콜 단일 출처화 목표 구조

11.2.3 3. 펌웨어 비차단화

tFORWARD 등 시간 지정 명령은 메인 루프를 점유하여, 그 동안 비상정지 (STOP_ALL)를 포함한 다른 명령 처리가 지연된다. 시간 명령을 타이머 기반 비차단 상태 기계로 전환하면 반응성과 안전성이 향상된다. 부저는 이미 비차단 패턴을 채택하고 있어 동일한 모델을 구동계로 확장할 수 있다.

11.2.4 4. 설정 정합성 관리

설정이 펌웨어 system.json과 웹 localStorage에 이중으로 존재한다. 연결 시 GET_SYS로 펌웨어 값을 신뢰 기준으로 동기화하는 흐름을 명확히 하고, 불일치 시 사용자에게 알리는 정책을 정립한다.

11.2.5 5. 안전성 · 견고성

- 장시간 운용 시 메모리 단편화를 고려한 주기적 gc.collect() 검토.
- 멈춘 명령에 대비한 위치독 · 타임아웃 정책.
- 빠른 연속 명령 전송 시의 웹 측 프라미스 경쟁 상태 점검.

11.2.6 6. AI 보조 기능 통합

문법 기반 NL→블록 파서(ai_helper*.js)와 로컬 LLM PoC (AI/ai.py)를 블록 편집기 UI에 정식 통합하여, 자연어 입력으로 블록을 생성하거나 코딩 질문에 답하는 도우미 패널을 제공한다. 오프라인 · 어린이 대상 특성을 고려해, 경량 문법 파서를 기본 경로로 두고 LLM은 선택적 보강으로 둔다.

11.2.7 7. 접근성 · 다국어

- OLED 한글 출력의 로마자 변환 한계를 보완할 한글 비트맵 폰트 검토.
- 블록 문구 · UI의 다국어(i18n) 분리로 해외 보급 대비.
- 색약 · 저시력 대비 카테고리 색상 대비 점검.

11.2.8 8. 테스트 · 빌드 자동화

- 명령 생성기·NL 파서에 대한 단위 테스트 도입.
- build.ps1/build.sh의 산출물 검증(스모크 테스트) 추가.
- 펌웨어 CommandProcessor에 대한 호스트 측 시뮬레이션 테스트.

우선순위 제언

파급효과 대비 비용을 고려하면, **(1) 통신 계층 추상화**와 **(2) 프로토콜 단일 출처화**가 가장 우선순위가 높다. 전자는 네이티브 확장의 전제이며, 후자는 두 코드베이스의 회귀를 구조적으로 차단한다.

주요 분석 요소 — 무엇부터 고칠 것인가

설계 의도

- 파급효과 대비 비용으로 개선 우선순위를 매겼다(전송 추상화·프로토콜 단일화가 최우선).

분석할 때 핵심 질문

- 회귀를 “구조적으로” 막는다는 말은 무슨 뜻인가?
- 어떤 개선이 다른 개선의 토대가 되는가?

점검 요소

- 개선 항목 하나를 골라 “변경 파일·필요한 테스트·예상 위험”을 담은 한 쪽짜리 계획서를 작성하라.

12 기술적 파급효과와 향후 추가 개발 계획

12.1 기술적 파급효과

12.1.1 교육적 파급효과

- **진입 장벽 완화**: 시각적 블록과 한국어 UI, 어린이용 단순화 인터페이스로 초등 저학년도 물리 로봇 제어를 경험한다.
- **실기 없는 학습 보장**: 동일 블록이 동일 명령으로 시뮬레이터와 실기 양쪽에서 동작하므로, 로버 보급 대수에 구애받지 않고 수업을 진행할 수 있다.
- **단계적 개념 습득**: 디지털 출력→다채널→난수→구동→센서→제어흐름→모듈화로 이어지는 미션 설계가 컴퓨팅 사고력을 점진적으로 형성한다.

12.1.2 기술적 파급효과

- **오프라인 우선 설계**: esbuild 번들과 fetch 심을 통한 file:// 단독 실행은, 네트워크·계정·설치가 제약되는 교실 환경에 적합한 배포 모델을 제시한다. 이는 다른 웹 기반 교구에 재사용 가능한 패턴이다.
- **개방형 하드웨어 구성**: system.json 기반 모듈·핀 런타임 구성은 동일 펌웨어로 다양한 하드웨어 조합을 수용하여, 저비용·맞춤형 교구 제작을 가능케 한다.
- **표준 웹 기술 활용**: Web Bluetooth·WebGL·Blockly 등 표준 기술만으로 구성되어 특정 벤더 종속이 없고, 유지보수·확장이 용이하다.

12.2 향후 추가 개발 계획

12.2.1 단기 (기존 자산 강화)

- 통신 계층 추상화 및 명령 프로토콜 단일 출처화(11장 우선순위 항목).
- AI 도우미(NL→블록, 코딩 질의응답)의 편집기 정식 통합.
- 펌웨어 시간 명령 비차단화로 비상정지 반응성 개선.
- 단위 테스트·빌드 스모크 테스트 도입.

12.2.2 중기 (플랫폼 확장)

- Electron 데스크톱 앱 출시(코드 재사용 최대).
- Android PWA 제공 및 Capacitor 래퍼 검토.
- iOS는 Capacitor + 네이티브 BLE 플러그인으로 대응.
- 미션·예제 콘텐츠 확장 및 교사용 진도·평가 도구 보강.

12.2.3 장기 (플랫폼 고도화)

- 다중 로버·협동 미션(다중 BLE 연결) 지원 검토.
- 센서·구동 모듈 확장(라인트레이서, 카메라 등)과 그에 맞는 블록·명령 추가.
- 학습 데이터·작업물 클라우드 동기화(선택적, 오프라인 우선 원칙 유지).
- 다국어 지원을 통한 해외 보급.
- 시뮬레이터의 물리·센서 모델 정교화로 실기와의 정합성 향상.

12.3 개발 로드맵 개관

위 계획을 단기·중기·장기로 배치하면 그림 15와 같다. 전송 추상화와 프로토콜 단일 출처화가 단기 토대가 되고, 이를 바탕으로 중기의 멀티플랫폼 확장과 장기의 플랫폼 고도화가 이어진다.

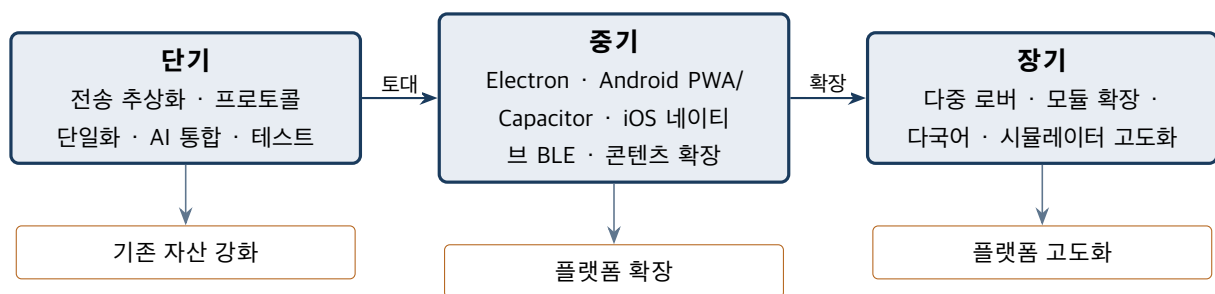


Figure 15. 단기 → 중기 → 장기 개발 로드맵

주요 분석 요소 — 이 설계가 남기는 것

설계 의도

- 오프라인 우선·표준 웹 기술·개방형 하드웨어 구성이 교육 보급성과 확장성을 만든다.

분석할 때 핵심 질문

- 이 설계의 어떤 점이 다른 교구에도 재사용될 수 있는가?
- 새 기능을 더할 때 “오프라인 우선” 원칙을 어떻게 지킬 것인가?

점검 요소

- 새 미션이나 새 하드웨어 모듈을 하나 제안하고, 1~12장 중 어느 장의 코드를 수정해야 하는지 매핑하라.

12.4 결론

ARES는 시각적 블록 코딩, 표준 웹 통신(Web Bluetooth), WebGL 3D 시뮬레이션, 모듈식 MicroPython 펌웨어를 결합하여, 초등 교육 현장에 적합한 오프라인 우선 로봇 코딩 플랫폼을 구현하였다. 코드베이스는 역할별로 명확히 분리되어 있으며, 웹과 펌웨어를 잇는 단순 텍스트 프로토콜이 시스템의 유연성과 확장성의 근간을 이룬다. 본 보고서가 분석한 구조적 특성과 개선·확장 계획은, ARES를 네이티브 멀티플랫폼으로 확장하고 콘텐츠·AI 보조 기능을 고도화하는 다음 단계의 기술적 토대가 된다.

13 코드 분석과 공동 개발 가이드

본 장은 앞선 분석(1~12장)을 실제 공동 개발로 잇기 위한 실무 가이드다. 코드를 어디부터 읽고, 어떻게 실행·디버그·빌드하며, 어떤 작업에 어떤 파일을 고쳐야 하는지를 한곳에 모은다.

13.1 코드 분석 진입점과 권장 읽기 순서

처음 코드를 여는 동료는 “큰 그림 → 제어 흐름 → 관심 영역”의 순서로 읽기를 권한다.

1. **큰 그림**: 1장(개요)과 9장(연계 구조)으로 세 도메인과 명령 계약을 파악한다.
2. **제어 흐름 따라가기**: 아래 두 진입점에서 호출 사슬을 손으로 따라간다.
3. **관심 영역 심화**: 펌웨어는 4장, 웹/블록은 5~7장, 3D/빌드는 8장, 세부 시그니처는 부록 A(API Reference)를 참조한다.

영역	읽기 진입점(호출 사슬)
펌웨어	main.py: AresRover.run() → _read_uart_line() → CommandProcessor.process() → 개별 핸들러 → robot(hardware.py)
웹	main.js: main() 초기화 → 실행 버튼 → CommandExecutor.executeWorkspace() → processBlock() → generateCommand() → sendCommand() → bluetooth.sendData()

13.2 개발 환경 구축

13.2.1 웹 UI

Listing 12. 웹 UI 로컬 실행

```
cd Web
```

```
python -m http.server 8000
# 브라우저에서 http://localhost:8000/main.html 열기
```

주의

개발 중에는 Web Bluetooth가 **보안 컨텍스트**(http://localhost 또는 HTTPS)에서만 동작하므로 소스를 file://로 바로 열면 BLE가 연결되지 않는다. 오프라인 단독 실행본(file://)은 8장의 빌드 절차로 만들어야 한다.

13.2.2 피코 펌웨어

Pico/의 모든 .py 파일을 Thonny·ampy·rshell 중 하나로 라즈베리파이 피코에 업로드한다. 시리얼 디버거는 9600 baud이며, 펌웨어는 print() 출력으로 부팅·모듈 초기화 상태를 보고한다. Pico/backup/과 날짜 백업 파일은 수정 대상이 아니다.

13.2.3 AI 모듈

AI/ai.py는 PyTorch + Hugging Face Transformers로 로컬 LLM (EXAONE-3.5-2.4B)을 구동하는 독립 PoC다. 별도의 파이썬 환경(GPU 권장)이 필요하며, 현재 웹 편집기와는 분리되어 있다(11장 통합 계획 참조).

13.3 변경 영향 지도 — 작업별 수정 파일

자주 발생하는 변경에 대해 함께 수정해야 하는 파일을 정리한다. 특히 웹과 펌웨어를 잇는 명령은 **양쪽**을 동시에 고쳐야 한다(9장 계약 규칙 참조).

작업	수정할 파일
새 Blockly 블록 추가	Web/blocklyconfig.js, Web/main.html(툴박스 XML), Web/commandexecutor.js
새 펌웨어 명령 추가	Pico/process_data.py(CommandProcessor, 접두 순서 유의)
웹·펌웨어 공용 새 명령	위 두 항목을 동시에, 응답 정책(fire-and-forget) 일치
기본 시스템 설정 변경	Pico/system_config.py(DEFAULT_CONFIG)
GPIO 핀 변경	Pico/pins.py(또는 런타임 SET_PIN)
하드웨어 모듈 추가/수정	Pico/<module>.py, Pico/hardware.py, Pico/process_data.py
BLE 타이밍/UUID 변경	Web/constants.js, Web/bluetooth.js, Pico/main.py(UART)
3D 모델/시뮬레이션 추가	Web/Mesh/, Web/simulation.js, 빌드 인라인(build.*)

13.4 디버깅 · 로깅 · 진단

수단	용도
웹 통신 로그	logger.js의 로그 UI — 송수신 명령·응답을 시간순으로 확인
대시보드	dashboard.html — 센서 텔레메트리, 수동 제어, 진단
브라우저 콘솔	예외·경고, state 전역 상태 점검
시리얼 모니터	펌웨어 print() 출력(9600 baud) — 부팅·모듈 init 실패 메시지

흔한 실패 지점과 점검 순서

- **BLE 연결 실패**: 보안 컨텍스트(localhost/HTTPS)인가? 사용자 제스처(버튼 클릭) 안에서 연결했는가? 장치 이름 필터·전원·거리 확인.
- **긴 명령이 깨짐**: 20바이트 청크 타이밍(CHUNK_DELAY)과 펌웨어 수신 버퍼(512B)·타임아웃을 확인.
- **명령이 무시됨**: 해당 모듈이 system.json에서 비활성인가? process_data.py의 접두 일치 순서가 맞는가?
- **응답이 멈춤**: fire-and-forget 목록이 웹·펌웨어 양측에서 일치하는가? 차단형 시간 명령(tFORWARD)이 루프를 점유하고 있지 않은가?
- **핀 변경이 안 먹힘**: SET_PIN 후 재부팅했는가? 핀 충돌·예비 핀 확인.

13.5 오프라인 빌드 · 배포와 기기 간 동기화

- **오프라인 빌드**: build.sh(macOS/Linux)·build.ps1(Windows)로 Web/을 Build/ 단일 폴더로 변환한다(8장 상세). 산출물은 file://로 더블클릭 실행한다.
- **버전관리**: 저장소는 클라우드 동기화 폴더(OneDrive 등) 밖에 clone 한다. OneDrive “파일 온디맨드”가 .git 객체를 손상시킨 사고가 있어, 기기 간 동기화는 git push/pull로만 한다.
- **배포 사본**: 빌드 결과는 저장소에 커밋하고, 배포용 OneDrive Build/ 폴더에도 복사해 최신으로 유지한다(.git 제외).

13.6 공동 개발 분석 체크리스트

새 기능을 분석·구현하기 전에 다음을 점검하면 회귀와 누락을 줄일 수 있다.

1. 이 변경은 웹·펌웨어 중 어느 쪽에 영향을 주는가? 명령 프로토콜을 건드리면 **양쪽**을 함께 바꿨는가?
2. 새 명령의 **응답 정책**(fire-and-forget 여부)을 양측에서 일치시켰는가?
3. 새 블록은 blocklyconfig.js·툴박스·commandexecutor.js 세 곳을 모두 갱신했는가?
4. 새 하드웨어는 모듈 활성 플래그·핀 할당·process_data.py 널 검사를 갖췄는가?
5. 긴 명령이라면 **BLE 청크·버퍼 한계**를 고려했는가?
6. 실기 없이 **시뮬레이터**로도 동작이 검증되는가?
7. 변경을 Document/API_DOCUMENTATION.md 등 **단일 출처 문서**에 반영했는가?

주요 분석 요소 — 분석을 개발로 잇기

설계 의도

- 이 가이드는 “코드를 이해한다”와 “코드를 바꾼다” 사이의 간극을 줄이기 위한 것이다. 진입점·영향 지도·체크리스트로 안전한 변경을 돕는다.

핵심 질문

- 내가 맡은 작업의 “변경 영향 지도”상 위치는 어디이며, 함께 바뀌어야 할 파일은 무엇인가?
- 변경을 어떻게 실행·시뮬·시리얼로 검증할 것인가?

점검 요소

- 첫 작업으로, 작은 변경(예: 블록 1개 추가) 하나를 진입점부터 검증까지 끝까지 수행하며 본 가이드의 각 단계를 실제로 밟아 보라.

14 연구 주제 도출과 논문 방향

본 장은 ARES를 단순한 도구 개발을 넘어 학술 연구로 발전시키기 위해, 앞선 분석(1~13장)에서 학술적 가치가 있는 요소를 추출하고 연구 요소별 연구 방향과 논문화 전략을 제시한다.

서술 원칙

본 장에서 제시하는 효과·수치는 아직 측정되지 않은 연구 가설이다. 실제 실험·측정으로 입증하기 전에는 성능·학습 효과를 단정하지 않으며, 모든 정량 주장은 “측정 대상”으로만 기술한다. 이는 연구 재현성과 진실성을 위한 것이다.

14.1 학술적 가치의 세 축

ARES의 코드·설계에는 서로 다른 분야의 연구 질문을 던질 수 있는 세 가지 가치 축이 있다.

1. **교육적 기여** — 블록 코딩 → 물리 로봇 제어, 시뮬레이터-실기 동형 설계, 오프라인 우선 접근성이 컴퓨팅 사고력·학습 동기에 주는 영향.
2. **시스템·공학적 기여** — 텍스트 명령 계약 기반 느슨한 결합, Web Bluetooth 통신 특성, 오프라인 단독 실행 빌드, 제약 환경 모듈식 펌웨어.
3. **AI·HCI 기여** — 오프라인·경량 자연어→블록 변환, 어린이 대상 블록 UI/UX와 3D 시뮬레이션의 동기 효과.

14.2 연구 요소 1 — 시뮬레이터-실기 동형 설계의 교육 효과

학술적 가치. ARES는 동일 블록이 동일 명령으로 시뮬레이터와 실기 양쪽에서 같은 의미로 동작한다(7·9장). 이 “동형성(isomorphism)”은 로버 보급 대수와 무관한 학습을 가능케 한다는 점에서 저자원 로보틱스 교육의 설계 원칙으로 검증할 가치가 있다.

연구 질문(RQ).

- RQ1. 시뮬레이터 선행 학습이 실기로의 전이(transfer)와 디버깅 수행에 미치는 효과는?
- RQ2. 로버 1:N(공유) 환경에서 동형 설계가 학습 성취 격차를 줄이는가?

연구 방법. 준실험 설계(시물-우선 / 실기-우선 / 혼합 집단), 사전-사후 컴퓨팅 사고력 평가, 미션 완수율·디버깅 소요 시간·오류 유형 분석.

기대 기여. 저자원·대규모 교실에서 “시물-실기 동형” 설계의 학습 효과에 대한 실증 근거.

논문 방향. 컴퓨터·정보 교육 분야 실증 연구(예: 컴퓨터교육·정보교육 계열 학술지). 논문 유형: 실증(empirical) 연구.

14.3 연구 요소 2 — 오프라인 우선 웹-임베디드 교구 아키텍처

학술적 가치. esbuild 번들링 + fetch 심 + 자산 인라인으로 file:// 단독 실행을 구현한 것(6·8장)은, 네트워크·설치·계정 없이 표준 웹 기술만으로 브라우저에서 임베디드 하드웨어를 제어하는 재현 가능한 배포 패턴이다.

연구 질문(RQ).

- RQ1. 오프라인 우선 설계가 교육 현장 배포 용이성·유지보수 비용에 주는 영향은?
- RQ2. 이 빌드 패턴(번들 + 심 + 인라인)은 다른 웹-임베디드 교구로 일반화되는가?

연구 방법. 설계 과학(Design Science) 또는 사례 연구 + 산출물 특성 실측(번들 용량, 외부 의존성 수, 브라우저·OS 호환 범위) + 현장 배포 사례 기술.

기대 기여. 저자원 환경을 위한 “오프라인 우선 웹-임베디드 교구” 레퍼런스 아키텍처와 일반화 가능한 빌드 패턴.

논문 방향. 교육공학·시스템 분야의 설계·사례 논문. 논문 유형: 설계 과학/시스템 기여.

14.4 연구 요소 3 — 브라우저-임베디드(Web Bluetooth) 통신 특성

학술적 가치. 20바이트 청크, 연결 인터벌 한계, 청크 지연, fire-and-forget 대 응답 대기, 차단형 시간 명령 등(4·6장) 웹 BLE 제어의 실측 특성과 교육 로봇 제어 적합성은 측정 기반 연구 대상이다.

연구 질문(RQ).

- RQ1. 청크 지연·페이로드 크기·응답 정책이 명령 처리율·패킷 손실·체감 지연에 주는 영향은?
- RQ2. 교육용 시나리오(연속 구동, 일괄 명령)에 대한 최적 파라미터는?

연구 방법. 다양한 CHUNK_DELAY·페이로드·기기·브라우저 조합에 대한 실측 벤치마크(처리율·손실률·왕복 지연). 모든 수치는 측정으로 확보하며 날조하지 않는다.

기대 기여. 교육용 웹 BLE 제어의 설계 가이드라인(파라미터 권고).

논문 방향. 임베디드·시스템·HCI 측정 연구. 논문 유형: 측정/벤치마크 연구.

14.5 연구 요소 4 — 제약 환경 모듈식 펌웨어와 결함 내성

학술적 가치. MicroPython RAM 제약 하에서 설정 주도 모듈 토글과 “설정 검사 → try/except → None 검사”의 3단 방어(2·4장)는 부분 고장·부분 구성에서도 운용을 보장한다. 단일 펌웨어로 다양한 하드웨어 구성을 수용하는 설계 패턴이다.

연구 질문(RQ).

- RQ1. 설정 주도 모듈식 펌웨어가 하드웨어 다양성 수용·결함 격리·교실 운용 신뢰성에 주는 효과는?
- RQ2. 3단 방어 패턴은 다른 교육용 임베디드 시스템으로 일반화되는가?

연구 방법. 결함 주입(모듈 제거·고장) 실험으로 부팅 성공률·기능 저하(degradation) 양상 측정, 구성 다양성 사례 분석.

기대 기여. 교육 로봇용 “구성 주도 결함 내성 펌웨어” 설계 패턴.

논문 방향. 임베디드 소프트웨어·소프트웨어공학(설계 패턴). 논문 유형: 설계 패턴/시스템 연구.

14.6 연구 요소 5 — 오프라인·경량 자연어→블록 변환

학술적 가치. 규칙·문법 기반 NL→Blockly 파서(if/else·while/until·중첩 제어)와 로컬 LLM PoC(1장, 부록 A2·A3)는 오프라인·경량 제약 하의 자연어→코드(NL2Code)와 어린이 한국어 의도 파싱이라는 연구 주제를 연다.

연구 질문(RQ).

- RQ1. 문법 기반 파서와 경량 LLM의 정확도·커버리지·지연 트레이드오프는?
- RQ2. 어린이 발화 분포(비표준 어순·구어체)에서의 강건성과 학습 스케폴딩 효과는?

연구 방법. 어린이 명령 코퍼스 구축, 파싱 정확도·커버리지 측정, 블록 생성 성공률과 만족도에 대한 사용자 연구.

기대 기여. 오프라인 교육용 NL2Block 벤치마크·방법론.

논문 방향. 교육 AI·자연어 처리(한국어). 논문 유형: 방법론 + 평가 연구.

14.7 인접 연구 주제(HCI)

추가로, 어린이 대상 블록 UI/UX(이모지 카테고리·단순화 컨텍스트 메뉴)와 WebGL 3D 시뮬레이션의 동기·몰입 효과는 HCI 사용성 연구로 확장 가능하다 (5·7·8장).

14.8 논문화 로드맵

연구 요소를 연구 유형·필요 데이터·대상 분야·우선순위로 정리하면 다음과 같다.

연구 요소	연구 유형	필요 데이터·측정	대상 분야	우선순위
1. 시물-실기 동형	실증	학습자 사전/사후, 미션 로그	CS·정보 교육	높음
2. 오프라인 우선 아키텍처	설계 과학	산출물 특성, 배포 사례	교육공학·시스템	높음
3. Web BLE 통신 특성	측정	통신 벤치마크 실측	임베디드·HCI	중간
4. 모듈식 결합 내성 펌웨어	설계 패턴	결합 주입 실험	임베디드 SW	중간
5. 오프라인 NL2Block	방법+평가	어린이 명령 코퍼스	교육 AI·NLP	중간

14.9 통합 논문 전략

- **대표(flagship) 논문:** “시물-실기 동형 + 오프라인 우선”을 결합한 시스템 + 교육 효과 논문. 시스템 기여(아키텍처)와 실증 기여(학습 효과)를 함께 제시해 영향력을 높인다(연구 요소 1·2 결합).
- **위성(satellite) 논문:** 통신 특성(요소 3), 결합 내성 펌웨어(요소 4), NL2Block(요소 5)을 각각 독립 논문으로 분리해 기여를 선명히 한다.

- **계재 순서 제안:** 측정·설계 기여(요소 2·3·4)는 데이터 확보가 비교적 빠르므로 먼저, 학습 효과(요소 1)와 코퍼스 기반(요소 5)은 피험자·데이터 수집 후 진행한다.

14.10 연구 타당성과 윤리

타당성·윤리 점검

- **어린이 피험자:** 학습 효과·사용자 연구는 보호자 동의와 기관 생명윤리(IRB) 승인 절차를 따른다.
- **측정 진실성:** 성능·학습 수치는 실제 측정으로만 보고하고, 미확보 수치는 본문·논문에 기재하지 않는다(재현성 원칙).
- **일반화 한계:** 단일 플랫폼·특정 연령·언어(한국어) 기반이라는 외적 타당도 한계를 명시한다.
- **재현성:** 코드·빌드 산출물·데이터를 공개해 재현 가능하도록 한다.

주요 분석 요소 — 분석에서 연구로

설계 의도

- 코드 분석으로 드러난 “설계 결정”을 연구 질문으로 바꾸는 것이 이 장의 목적이다. 구현 특징(동형성·오프라인·체크·모듈식)을 그대로 검증 대상으로 삼는다.

핵심 질문

- 우리가 맡은 구현 요소 중 “측정하면 학술적 주장이 되는” 것은 무엇인가?
- 그 주장을 입증하려면 어떤 데이터·실험·대조군이 필요한가?

점검 요소

- 연구 요소 1·5 중 하나를 골라 RQ-방법-측정 지표-필요 데이터를 한 쪽 분량의 연구 계획서로 구체화하라(IRB·측정 진실성 원칙 포함).

A1 Pico 소스 코드별 API Reference

본 부록은 Pico/ 디렉터리 MicroPython 소스의 클래스·메서드·함수 시그니처를 파일별로 정리한다. 시그니처는 소스에서 그대로 옮긴 것이며, backup/ 및 날짜 백업 파일은 제외한다.

A1.1 main.py

모듈 상수: UART_ID=0, UART_BAUDRATE=9600, MAIN_LOOP_DELAY_MS=10, RECEIVE_TIMEOUT_MS=500, MAX_BUFFER_SIZE=512.

class AresRover — UART 통신과 명령 처리를 총괄하는 메인 컨트롤러.

시그니처	설명
def __init__(self):	UART/명령처리기/하드웨어 초기화
def boot(self):	안전 정지·부팅음·OLED 메시지 부팅 시퀀스
def _safe_stop_all_motors(self):	전체 모터 비상 정지
def _pop_line(self):	버퍼에서 개행 단위 완성 라인 추출(없으면 None)
def _read_uart_line(self):	타임아웃 허용 다중 청크 UART 라인 수신
def _needs_response(self, data):	응답 필요 여부 판단(fire-and-forget 필터)
def _is_status_message(self, data):	'+' 시작 BT 상태 메시지 여부
def _send_response(self, response):	20바이트 청크로 응답 전송
def _process_command(self, data):	명령 처리 후 결과 문자열 반환
def run(self):	명령 수신·부저 갱신 메인 루프

A1.2 process_data.py

모듈 상수: PWM_MAX=65535.

class CommandProcessor — UART 명령 파서 및 디스패처.

시그니처	설명
def process(self, data):	명령을 핸들러로 분배하는 메인 디스패처
def _get_speed_pwm(self):	max_speed로부터 PWM 값 계산
def _handle_ping(self):	연결 확인 + OLED 갱신
def _handle_stop_all(self):	비상 정지 + OLED 메시지
def _handle_get_status(self):	거리·자기·온도·메모리·업타임 반환
def _handle_get_sys(self):	시스템 설정값 반환
def _handle_get_modules(self):	활성 모듈 정보 반환
def _handle_set_pin(self, data):	핀 할당 변경(SET_PIN, 이름, 번호)

시그니처	설명
<code>def _handle_set_module(self, data):</code>	모듈 활성화/비활성(<code>SET_MODULE</code> , 이름,0/1)
<code>def _handle_batch(self, data):</code>	파이프 구분 배치 실행(<code>BATCH;cmd1 cmd2</code>)
<code>def _handle_sys_set(self, data):</code>	시스템 설정 저장(<code>SYS_SET,...</code>)
<code>def _handle_buzzer_on(self, data):</code>	비차단 부저 시작(<code>BUZZER_ON</code> ,주파수, 지속)
<code>def _handle_sing(self):</code>	멜로디 재생(<code>Hala Madrid</code>)
<code>def _handle_timed_wheel(self, data, direction):</code>	방향별 시간 지정 서보 이동
<code>def _handle_continuous_wheel(self, direction):</code>	연속 서보 이동
<code>def _handle_dcmotor(self, data):</code>	레거시 DC 모터 제어
<code>def _handle_timed_dcmotor(self, data):</code>	레거시 시간 지정 DC 모터
<code>def _handle_timed_dcmotor_new(self, data, direction):</code>	신규 형식 시간 지정 DC 모터
<code>def _handle_main_motor(self, direction):</code>	연속 DC 모터 구동
<code>def _handle_calib_start(self):</code>	보정 테스트 시퀀스 실행
<code>def _handle_calib_set(self, data):</code>	보정값 저장(<code>CALIB_SET</code> ,좌,우)
<code>def _handle_led_on(self, data):</code>	LED 켜기(<code>LED_ON</code> ,번호,밝기)
<code>def _handle_led_off(self, data):</code>	LED 끄기(<code>LED_OFF</code> ,번호 ALL)
<code>def _handle_led_pattern(self, data):</code>	LED 패턴 적용([v0 v1 ... v5])
<code>def _handle_msg(self, data):</code>	OLED 텍스트(16자 자동 줄바꿈)
<code>def _handle_clear_rect(self, data):</code>	사각 영역 지우기(<code>CLEAR_RECT</code> ,x,y,w,h)
<code>def _handle_msg_xy(self, data):</code>	좌표 텍스트(<code>MSG_XY</code> ,x,y,text)
<code>def _handle_icon(self, data):</code>	아이콘 표시(<code>ICON</code> ,name,x,y)
<code>def _handle_sleep(self, data):</code>	대기(<code>SLEEP</code> ,초)

A1.3 hardware.py

class RobotHardware — 모듈 선택 로딩을 수행하는 하드웨어 싱글턴. 전역 인스턴스 `robot = RobotHardware()`.

시그니처	설명
<code>def __new__(cls):</code>	싱글턴 인스턴스 보장
<code>def _init_hardware(self):</code>	설정에 따른 전체 모듈 초기화
<code>def _sync_bt_name(self):</code>	설정의 블루투스 이름 동기화

시그니처	설명
<code>def get_temperature(self):</code>	내장 온도 센서 읽기
<code>def get_battery_voltage(self):</code>	배터리 전압(미구현, 100 반환)
<code>def get_status_dict(self):</code>	센서값·지표 상태 딕셔너리 반환
<code>def stop_all(self):</code>	모터·LED·부저·발사기 전체 정지

A1.4 system_config.py

모듈 상수: `CONFIG_FILE='system.json'`, `LEGACY_CALIB_FILE='calibration.json'`, `MAX_DEVICE_NAME_LENGTH=10`, `DEFAULT_CONFIG`, `CONFIG_KEY_ORDER`.

class SystemConfig — 파일 영속화 설정 싱글턴. 전역 인스턴스 `sys_config = SystemConfig()`.

시그니처	설명
<code>def __new__(cls):</code>	싱글턴 인스턴스
<code>def _init_config(self):</code>	설정 초기화 및 파일 로드
<code>def load_config(self):</code>	system.json 로드(레거시 보정 파일 이관 포함)
<code>def save_config(self, max_speed, collision_dist, auto_stop, device_name):</code>	시스템 설정 저장
<code>def save_calibration(self, left, right):</code>	휠 보정 계수 저장
<code>def save_all(self):</code>	순서화 키로 전체 설정 저장
<code>def get(self, key):</code>	설정값 조회
<code>def set_pin(self, pin_name, pin_number):</code>	핀 할당 변경(0~28 검증)
<code>def enable_module(self, module_name, enable):</code>	모듈 활성화/비활성
<code>def is_module_enabled(self, module_name):</code>	모듈 활성화 여부
<code>def get_module_info(self):</code>	모듈 상태·설명 반환

A1.5 pins.py

모듈 상수:

```
UART_TX_PIN=0    UART_RX_PIN=1    DCMOTOR_DIR_PIN=8    DCMOTOR_PWM_PIN=9
I2C_SDA_PIN=4    I2C_SCL_PIN=5    ULTRASONIC_TRIG_PIN=6    ULTRASONIC_ECHO_PIN=7
SERVO_RIGHT_PIN=12    SERVO_LEFT_PIN=13    BUZZER_PIN=15    GUN_PIN=22
MAGSENSOR_PIN=26    LED_PINS=[21,20,19,18,17,16]    RESERVED_PINS=[10,11,14,27,28]
```

함수: `def print_pin_info():` — 핀 할당 표를 콘솔 출력.

A1.6 wheel.py

class KSWheel — 보정 지원 서보 휠 컨트롤러.

시그니처	설명
<code>def __init__(self):</code>	서보 중립 듀티·보정 계수 초기화
<code>def update_factors(self, left, right):</code>	보정 계수 갱신(최대 100)
<code>def set_angle(self, wheel_idx, angle, speed=1.0):</code>	각도(0~180)→PWM 듀티 매핑
<code>def forward(self, speed=0.1):</code>	전진
<code>def backward(self, speed=0.1):</code>	후진
<code>def turn_right(self, speed=0.1):</code>	우회전
<code>def turn_left(self, speed=0.1):</code>	좌회전
<code>def stop(self):</code>	양 휠 정지(중립)

A1.7 dcmotor.py

모듈 상수: PWM_FREQUENCY=20000, PWM_STOP=0, PWM_MAX=65535, DEBUG_MOTOR=False.

class KSDCMotor — 브레이크 변조 듀얼 PWM H-브리지 DC 모터.

시그니처	설명
<code>def __init__(self):</code>	IN1·IN2 양 핀 PWM 초기화
<code>def dc_forward(self, speed):</code>	전진(IN2를 PWM_MAX-speed로 변조)
<code>def dc_backward(self, speed):</code>	후진(역방향 변조)
<code>def dc_stop(self):</code>	정지(코스트)
<code>def is_running(self):</code>	구동 상태 반환

A1.8 buzzer.py

class KSBuzzer — 차단/비차단 재생 부저.

시그니처	설명
<code>def __init__(self, pin_num=None):</code>	부저 PWM 초기화(기본 BUZZER_PIN)
<code>def play(self, freq=1000, duration=1, vol=1000):</code>	완료까지 차단 재생
<code>def start(self, freq=1000, duration=1, vol=1000):</code>	비차단 시작(재생 중이면 False)
<code>def update(self):</code>	지속시간 경과 시 자동 정지
<code>def stop(self):</code>	즉시 정지
<code>def boot_sound(self, short=False):</code>	부팅음(G5 0.2s + C4 0.9s)
<code>def halamadrid(self, short=False):</code>	18음 멜로디 재생

A1.9 leds.py

class KSLEDs — 패턴 지원 LED 배열 컨트롤러.

시그니처	설명
<code>def __init__(self):</code>	6개 LED 20kHz PWM 초기화 후 소등
<code>def leds_off(self):</code>	전체 소등
<code>def swipe_effect(self):</code>	스와이프 애니메이션(100ms 타이밍)
<code>def check(self):</code>	시작 점검 패턴(3패스, 페이딩)
<code>def set_led_pattern(self, pattern):</code>	밝기 패턴 적용(0.0~1.0 리스트)

A1.10 gun.py

class KSGun — BB탄 발사기 컨트롤러.

시그니처	설명
<code>def __init__(self, pin_num=None):</code>	발사 모터 PWM 1kHz 초기화(기본 GUN_PIN)
<code>def power(self, value):</code>	PWM 듀티 설정
<code>def stop(self):</code>	모터 정지
<code>def soft_start(self, target_power, duration_ms=150):</code>	8단계 파워 램프업
<code>def fire_once(self, power=50000, spin_time=0.22, cooldown_ms=250):</code>	단발 발사(램프업→회전→정지)

A1.11 ultrasonic.py

class KSDistance — HC-SR04 초음파 거리 센서.

시그니처	설명
<code>def __init__(self):</code>	트리거(OUT)·에코(IN) 핀 초기화
<code>def get_distance(self):</code>	cm 거리 측정(30ms 타임아웃, 오류 시 1000000)

A1.12 magsensor.py

class KSMagSensor — 자기장 센서(액티브 로우).

시그니처	설명
<code>def __init__(self):</code>	MAGSENSOR_PIN 폴업 입력 초기화
<code>def detect(self):</code>	자석 검출(0→검출=1, 1→미검출=0 반환)

A1.13 ssd1306.py

SSD1306 명령 레지스터 상수(SET_CONTRAST=0x81 등) 다수 정의.

class SSD1306 — FrameBuffer 기반 OLED 베이스 컨트롤러.

시그니처	설명
<code>def __init__(self, width, height, external_vcc):</code>	프레임버퍼·아이콘 초기화
<code>def init_display(self):</code>	초기화 명령 시퀀스 전송
<code>def poweroff(self): / def poweron(self):</code>	디스플레이 전원 제어
<code>def contrast(self, contrast):</code>	명암 설정(0~255)
<code>def invert(self, invert):</code>	색 반전
<code>def show(self):</code>	프레임버퍼 플러시
<code>def fill(self, col):</code>	전체 채우기(0/1)
<code>def pixel(self, x, y, col):</code>	픽셀 설정
<code>def fill_rect(self, x, y, w, h, col):</code>	사각 영역 채우기
<code>def scroll(self, dx, dy):</code>	프레임버퍼 스크롤
<code>def text(self, string, x, y, col=1):</code>	텍스트 그리기
<code>def booting_msg(self):</code>	부팅 스플래시 표시

class SSD1306_I2C — I2C 인터페이스 변형.

시그니처	설명
<code>def __init__(self, width, height, scl_pin=None, sda_pin=None, addr=0x3c, external_vcc=False):</code>	I2C 초기화
<code>def write_cmd(self, cmd):</code>	I2C 명령 바이트 쓰기
<code>def write_data(self, buf):</code>	I2C 데이터 버퍼 쓰기

class SSD1306_SPI — SPI 인터페이스 변형(현재 미사용). `def __init__(self, width, height, spi, dc, res, cs, external_vcc=False):`, `write_cmd(self, cmd)`, `write_data(self, buf)`.

A1.14 icon.py

모듈 상수(비트맵): `mars_rover32x32`, `cute_robot32x32`, `open_eye32x32`, `closed_eye32x32` (각 32×32).

class KSicon — 수평 바이트 비트맵을 MONO_VLSB 수직 바이트로 변환·렌더.

시그니처	설명
<code>def __init__(self, buff, w, h, oled):</code>	비트맵 포맷 변환
<code>def blit(self, x, y):</code>	지정 좌표에 아이콘 렌더

A2 Web 소스 코드별 API Reference

본 부록은 Web/ 디렉터리 JavaScript 소스(최상위, vendor/·Mesh/ 제외)의 공개 객체·함수 시그니처를 파일별로 정리한다.

A2.1 constants.js

설정 객체를 export 한다.

- `BLUETOOTH_CONFIG` — { `UART_SERVICE_UUID`, `UART_CHARACTERISTIC_UUID`, `MAX_CHUNK_SIZE`, `COMMAND_DELAY`, `CHUNK_DELAY`, `READ_INTERVAL`, `RESPONSE_TIMEOUT` }
- `DEFAULT_SYSTEM_CONFIG` — { `device_name`, `max_speed`, `collision_distance`, `auto_stop_enabled`, `left_calibration`, `right_calibration`, `connection_timeout`, `reconnect_attempts`, `chunk_size`, `command_delay` }
- `STATUS_COLORS` — { `GREEN`, `RED`, `ORANGE` }
- `STORAGE_KEYS` — { `SYSTEM_CONFIG` }

A2.2 state.js

- `DEBUG` — 상세 로깅 플래그(boolean).
- `state` — 전역 상태 객체. 블루투스 핸들(`bluetoothDevice`, `bluetoothServer`, `uartService`, `characteristic`, `notificationsEnabled`, `readIntervalId`, `isConnecting`, `connectFailed`, `lastCommand`), 실행(`isExecuting`), 변수 맵(`variables`), 프라미스 상태(`pendingResolve`, `pendingReject`, `pendingTimeout`).

A2.3 elements.js

`elements` 객체로 DOM 참조 캐시: `connectButton`, `runButton`, `saveButton`, `loadButton`, `fileInput`, `logContent`, `logContainer`, `clearLogBtn`.

A2.4 logger.js

export 모듈 `Logger`.

시그니처	설명
<code>Logger.add(message, type = 'info', options = {})</code>	로그 항목 추가(type, verbose/detail)
<code>Logger.clear()</code>	전체 로그 비우기

시그니처	설명
<code>Logger.refresh()</code>	로그 표시 재렌더

A2.5 romanize.js

시그니처	설명
<code>romanizeKorean(input)</code>	한글→로마자 변환(OLED용)
<code>hasKorean(input)</code>	한글 포함 여부 반환(boolean)

A2.6 blocklyconfig.js

- `BATCH_FORBIDDEN_TYPES` — `batch_block` 내부 금지 블록 타입 Set.
- `BlocklyConfig` — blocks 배열(블록 JSON 정의).
- `attachBatchBlockValidator(BlocklyLib)` — `batch_block`에 `onchange` 검증기 부착.

A2.7 bluetooth.js

export 모듈 `BluetoothManager`.

시그니처	설명
<code>async connect()</code>	BLE 장치 페어링 · 서비스 탐색
<code>async disconnect()</code>	연결 종료 · 정리
<code>async cleanup()</code>	리스너 · 타이머 · 버퍼 정리
<code>onDeviceDisconnected()</code>	비정상 연결 해제 처리
<code>handleRxData(event)</code>	수신 알림 처리
<code>processReceivedData(receivedData)</code>	수신 응답 파싱 · 분배
<code>_handleSysValues(data)</code>	SYS_VALUES 응답 파싱
<code>_handleCalibValues(data)</code>	CALIB_VALUES 응답 파싱
<code>_resolvePromise(data)</code>	대기 프라미스 해소
<code>_updateBlocklyVariable(data)</code>	DIST/MAG 추출 후 변수 갱신
<code>async readCharacteristic()</code>	단일 특성 읽기(폴링 폴백)
<code>startPeriodicReads()</code>	폴링 타이머 시작
<code>updateConnectionStatus(connected)</code>	연결 UI · 이벤트 갱신
<code>updateStatus(message, _color)</code>	상태 메시지 로깅(폐기 예정)
<code>async sendData(data, waitForResponse = false)</code>	20바이트 청크 전송, 응답 반환

시그니처	설명
delay(ms)	프라이미스 슬립 유틸

A2.8 commandexecutor.js

export 모듈 CommandExecutor.

시그니처	설명
FIRE_AND_FORGET_HEADS	응답 불필요 명령 헤드 Set
_isFireAndForget(command)	응답 불필요 여부 판단
async _dispatch(command, waitForResponse)	BLE/시뮬레이션 라우팅
_parseSensorReply(data)	센서값 추출·캐시
evaluateValueBlock(block)	값 블록 재귀 평가
async processBlock(block)	단일 블록 실행 후 다음 블록 재귀
async _processBatch(block)	batch_block 평탄화 후 BATCH 전송
generateCommand(block)	블록 타입→명령 문자열
async sendCommand(command)	쿨다운 포함 단일 명령 전송
async handleLogicBlock(block)	변수·반복·조건·함수 등 처리
_findProcedureDefinition(workspace, name, hasReturn)	함수 정의 블록 탐색
async _setupProcedureArgs(callBlock, defBlock)	호출 인자→지역변수 바인딩
async executeWorkspace(workspace)	워크스페이스 전체 실행
async simulateWorkspace(workspace, sink)	명령을 sink로 보내 시뮬레이션

A2.9 ai_helper.js

규칙 기반(오프라인) 자연어→Blockly XML 변환.

- parse(rawText) — { ok, replace, xml, added, unmatched, error, suggest } 반환.
- _internal — grammar.js 재사용용 내부 함수 (splitClauses, splitMeasureBoundary, matchAction, parseClause, detectComparison, detectOutputVar, KNOWN_TYPES).

A2.10 ai_helper_grammar.js

재귀 하강 파싱 기반 자연어→Blockly XML(if/else, while/until, 중첩 지원).

- parse(rawText) — ai_helper.parse와 동일 형식 반환.
- _grammar — 내부 함수(parseInline, parseClauseAt, parseProgram, matchRepeatHeader).

A2.11 simulation.js

three.js 기반 3D 시뮬레이션 초기화·관리.

- `setupSimulation({ workspace, onOpen, onClose })` — 시뮬레이션 UI 핸들러 등록 후 컨트롤러 객체 `{ close() }` 반환.
- 내부 헬퍼(`OLED_ICONS`, `TOPICS`, `buildSim`, `recolorLaunchpadAntenna` 등)는 공개 API 아님.

A2.12 main.js

애플리케이션 진입점. 라우팅·뷰 전환·Blockly 초기화·이벤트 리스너·버튼 콜백을 담당하며 대부분 내부 함수이다. 페이지 로드 시 `main()` 이 1회 호출되어 Blockly·이벤트·내비게이션·뷰를 초기화한다.

A2.13 mobile-preview.js

자기실행 IIFE. 공개 export 없음. `?mobile=true`일 때 페이지를 모바일 기기 프레임(iframe)으로 감싸 반응형 미리보기를 제공하며, 내부 링크에 `mobile=true&framed=1`을 전파한다.

A3 AI 소스 코드별 API Reference

본 부록은 AI/ 디렉터리의 소스(`ai.py`)를 정리한다. 브라우저에서 동작하는 문법 기반 자연어 파서(`ai_helper.js`, `ai_helper_grammar.js`)는 Web/에 위치하므로 부록 A2에 수록한다.

A3.1 ai.py

역할: EXAONE-3.5-2.4B 로컬 LLM을 래핑하여, 초등학생의 블록 코딩·로봇 조작 질문에 한국어로 답하는 코드 도우미 PoC.

모델·생성 설정

- `model_name = "LGAI-EXAONE/EXAONE-3.5-2.4B-Instruct"`
- 로딩: `dtype=torch.bfloat16, trust_remote_code=True, device_map="auto"`
- 스트리밍 생성: `max_new_tokens=2048, temperature=0.6, top_p=0.95, do_sample=True`
- 비스트리밍 생성: `max_new_tokens=256, temperature=0.6, top_p=0.95, do_sample=True`

class CodeAssistant — 어린이 대상 블록 코딩 질의응답 도우미. 속성: `self.model`, `self.tokenizer`, `self.device`.

시그니처	설명
<code>def __init__(self, model, tokenizer, device="cuda"):</code>	모델·토큰라이저·디바이스 설정
<code>def ask_about_code(self, user_query):</code>	질문 처리·응답 생성 (스트리밍/비스트리밍)

모듈 함수: `def call_ai(prompt):` — `CodeAssistant`를 인스턴스화하여 질의를 위임하는 편의 함수.

시스템 프롬프트: 모델이 초등학생 대상 라즈베리파이 로봇 조작 튜터로서, Google Blockly 블록 코딩을 쉬운 한국어로 설명하도록 지시한다. 로봇 제어 블록(램프·메시지·상태·거리/자기 센서·부저·이동·정지)과 제어 블록(대기·반복·변수·기초 연산)을 안내한다.

외부 의존성: torch(bfloat16), transformers.AutoModelForCausalLM, transformers.AutoTokenizer, transformers.TextIteratorStreamer, threading.Thread(스트리밍 생성), pathlib.Path.

— 부록 끝 —